计算几何

Morning Flower and Evening Oath

目录

第一章 二维计算几何基础 **1**

1.1 浮点点向量、圆与交点结果 . . . 1
1.2 点在直线上的投影 . . . 3
1.3 浮点两直线交点 . . . 4
1.4 浮点直线与圆交点 . . . 6
1.5 浮点两圆交点 . . . 8
1.6 两圆交面积 . . . 11
1.7 凸多边形裁剪：保留有向直线左侧 . . . 12
1.8 简单多边形面积重心 . . . 14
1.9 三角形内切圆：边长加权内心 . . . 15
1.10 有限二进制浮点极角精确排序 . . . 16
1.11 整数点向量与线段判定 . . . 19
1.12 整数多边形有向面积 . . . 22
1.13 点在整数简单多边形内 . . . 23
1.14 整数输入直线与圆交点 . . . 24
1.15 整数输入两圆交点 . . . 26
1.16 圆与简单多边形相交面积 . . . 28
1.17 两圆公切线与过点切线 . . . 30
1.18 一般多边形面积并：整数精确判定 . . . 32
1.19 圆面积并：未覆盖圆弧积分 . . . 39
1.20 圆覆盖重数面积：恰好覆盖 k 次 . . . 44

第二章 三维计算几何基础 **49**

2.1 三维凸包：精确方向与退化处理 . . . 49
2.2 整数三维共线、共面与方向 . . . 60
2.3 实数三维向量与夹角 . . . 61
2.4 三维直线投影、线段距离与绕轴旋转 . . . 65
2.5 三维平面投影与分类求交 . . . 68

第三章 距离 **71**

3.1 点到闭线段的距离 . . . 71

第四章 Pick 定理 **73**

4.1 Pick 定理：格点、剖分与孔洞 . . . 73

第五章 凸包 **74**

5.1 凸多边形外点的两条切线 . . . 74
5.2 点在整数凸包内（二分） . . . 77
5.3 整数凸包：单调链 . . . 78
5.4 凸多边形 Minkowski 和 . . . 79

第六章 扫描线 **82**

6.1 整数轴对齐矩形面积并 . . . 82

第七章 旋转卡壳 **84**
7.1 整数凸包直径：旋转卡壳 84
7.2 最小面积包围矩形：旋转卡壳与四角 86

第八章 半平面交 **89**
8.1 整数半平面的有界多边形交 89

第九章 平面最近点对 **93**
9.1 浮点平面最近点对 93

第十章 随机增量法 **96**
10.1 随机增量最小覆盖圆 96

第十一章 配套知识与验证范围 **99**

第十二章 计算几何：精确判定与适用范围 **100**
12.1 定点凸包、直径和极角排序 100
12.2 整数系数的有界半平面交 100
12.3 最近点对、Minkowski 和与三维整数判定 100
12.4 浮点最近点对 100
12.5 公切线与过点切线 100
12.6 随机增量最小覆盖圆 101
12.7 圆与简单多边形相交面积 101
12.8 最近点对编号与多边形闭集查询 101
12.9 整数圆交点：精确判定与浮点构造 101
12.10 公切线：切点次序也需要精确判定 101
12.11 圆交点：容差分类与坐标误差 102
12.12 三维投影、旋转与平面求交的数值约定 102
12.12.1 原三维片段中需要避开的错误 102

第一章　二维计算几何基础

1.1　浮点点向量、圆与交点结果

功能：浮点二维几何的公共基础：点/向量 `Point`（加减、数乘、数除），圆 `Circle`，以及各求交函数共用的结果类型 `Result`（交点类别 `Kind` 加有限个交点）。

核心思路：坐标统一用 `long double`。`dot`、`cross` 为点积与叉积，`cross(a,b)` > 0 表示 b 在 a 的逆时针方向；`norm` 用 `hypotl` 求长度，避免先平方溢出；`perp` 把向量逆时针旋转 $90°$。其余实数几何函数都建立在这些运算上，交点无穷多或输入退化等情况通过 `Kind` 区分。

接口契约：R 为 long double；Point 支持向量加减、数乘和数除（除数非零）。dot/cross/norm/perp 为基本运算，norm 使用 hypotl；Circle 保存圆心和非负半径。Kind 为 none、one、two、infinite、degenerate；Result 同时保存类别与有限交点。输入须有限，尺度须使后续平方、乘积不溢出或下溢；排序不使用 epsilon。具体容差与退化约定见各函数。

```
struct RealPlane
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0;

        Point operator+(Point b) const { return {x + b.x, y + b.y}; }

        Point operator-(Point b) const { return {x - b.x, y - b.y}; }

        Point operator*(R k) const { return {x * k, y * k}; }

        Point operator/(R k) const { return {x / k, y / k}; }
    };

    struct Circle
    {
        Point o;
        R r;
    };
```

```
    enum class Kind
    {
        none,
        one,
        two,
        infinite,
        degenerate
    };

    struct Result
    {
        Kind kind;
        vector<Point> p;
    };

    static R dot(Point a, Point b) { return a.x * b.x + a.y * b.y; }

    static R cross(Point a, Point b) { return a.x * b.y - a.y * b.x; }

    static R norm(Point a) { return hypotl(a.x, a.y); }

    static Point perp(Point a) { return {-a.y, a.x}; }
};
```

模板题使用：AOJ CGL_2_A

题意：每组给两条非退化直线，平行输出 2，垂直输出 1，否则输出 0。输入为绝对值不超过 10000 的整数，点积与叉积在此范围可由浮点精确表示，所以直接比较 0；这不适用于一般实数或更大整数。只演示点向量与 dot/cross，不涵盖圆及 Result 分类。

所需模板：RealPlane（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto u = b - a, v = d - c;
        if (RealPlane::cross(u, v) == 0)
            cout << 2 << '\n';
        else if (RealPlane::dot(u, v) == 0)
            cout << 1 << '\n';
        else
            cout << 0 << '\n';
    }
}
```

1.2 点在直线上的投影

功能： 求点 p 在过 a、b 两点的直线上的正交投影（垂足）。

核心思路： 设方向 $v = b - a$，投影为 $a + v \cdot \frac{(p-a)\cdot v}{v\cdot v}$，即从 a 出发沿 v 走 $p - a$ 在 v 上的投影长度。$v \cdot v = 0$（两点重合）时直接返回 a。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约： line_projection(p,a,b) 返回 p 在过 a、b 直线上的正交投影；若 a=b，则约定返回 a。类型为 RealPlane::Point。输入有限且尺度适当，平方和与点积须能可靠表示；不使用 epsilon 把短线段自动视为零。O(1) 时间和空间。

```
RealPlane::Point
line_projection(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return a;
    return a + v * (G::dot(p - a, v) / d);
}
```

模板题使用：AOJ CGL_1_A

题意：固定直线两端点，对每个点输出其在无限直线上的投影。题目保证两端点不同，整数坐标绝对值不超过 10000，q 不超过 1000；投影可在线段外。接口参数为 (p,a,b)，不截断到线段；a=b 时模板返回 a 属于额外边界。

所需模板：RealPlane（第 1 页，1.1 节）、line_projection（第 3 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_1_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point a, b;
    int q;
    cin >> a.x >> a.y >> b.x >> b.y >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point p;
        cin >> p.x >> p.y;
        auto h = line_projection(p, a, b);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

1.3 浮点两直线交点

功能：求直线 ab 与直线 cd 的交点，并区分唯一交点、平行不交、重合（无穷多交点）和直线退化四种情况。

核心思路：设 $u = b - a$，$v = d - c$，$\Delta = u \times v$。若 $|\Delta| \le \varepsilon |u||v|$（两方向夹角的正弦足够小）视为平行，再看 c 到直线 ab 的距离 $|(c-a) \times u|/|u|$ 是否在相对容差内，区分重合与平行不交；否则交点为 $a + u \cdot \frac{(c-a)\times v}{\Delta}$。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_intersection_real(a,b,c,d,eps=1e-12L) 返回 RealPlane::Result；eps 必须为正。端点重合使任一直线退化时返回 degenerate。按方向叉积与方向长度乘积的相对阈值判断平行，再按位置阈值区分 none 与 infinite；唯一交点返回 one。输入有限且尺度适当；近似分类不等同于精确共线判定。O(1)。

```
RealPlane::Result line_intersection_real(RealPlane::Point a,
                                         RealPlane::Point b,
                                         RealPlane::Point c,
                                         RealPlane::Point d,
                                         long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps > 0);
    Point u = b - a, v = d - c;
    R nu = G::norm(u), nv = G::norm(v);
    if (nu == 0 || nv == 0) return {Kind::degenerate, {}};
    R det = G::cross(u, v);
    if (fabsl(det) <= eps * nu * nv)
        return {fabsl(G::cross(c - a, u)) <= eps * nu * max(R(1), G::norm(c - a))
                    ? Kind::infinite
                    : Kind::none,
                {}};
    return {Kind::one, {a + u * (G::cross(c - a, v) / det)}};
}
```

模板题使用：AOJ CGL_2_C

题意：输出两条线段的唯一交点；题目保证非平行且相交，q 不超过 1000，整数坐标绝对值不超过 10000，因此可直接取 one 结果。模板计算支撑直线交点，不自动检查交点在线段内；一般输入须先检查 kind。整数方向叉积非零时绝对值至少 1，大于本题范围下默认相对阈值。

所需模板：RealPlane（第 1 页，1.1 节）、line_intersection_real（第 4 页，1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b, c, d;
        cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y >> d.x >> d.y;
        auto ans = line_intersection_real(a, b, c, d);
        assert(ans.kind == RealPlane::Kind::one);
        cout << ans.p[0].x << ' ' << ans.p[0].y << '\n';
    }
}
```

1.4 浮点直线与圆交点

功能：求直线 ab 与圆的交点，结果分为相离、相切（一点）、相交（两点）以及直线退化。

核心思路：分两种模式。eps= 0 时只用平方量：$v = b - a$，$z = v \times (a - o)$，圆心到直线距离为 $|z|/|v|$，判别式 $D = r^2|v|^2 - z^2$ 的符号决定交点个数；垂足 $h = o + \mathrm{perp}(v)\, z/|v|^2$，两交点为 $h \mp v\sqrt{D}/|v|^2$。eps> 0 时用 line_projection 求圆心垂足 h 及距离 d，与半径按相对容差比较，近切时合并为一点，否则从 h 沿直线方向偏移半弦长 $\sqrt{r^2 - d^2}$。

代码依赖：RealPlane（第 1 页，1.1 节）、line_projection（第 3 页，1.2 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_circle_intersections(a,b,c,eps=1e-12L) 返回 Result；c 为 RealPlane::Circle，半径非负，eps 非负；以下相对阈值描述适用于 eps>0。a=b 返回 degenerate；按投影点到圆心的距离和半径的相对阈值区分 none、one、two。点圆允许；只有正半径同心圆才涉及无限交点，本函数不会返回 infinite。依赖 line_projection，O(1)。输入有限且尺度适当，接近相切时按容差归类。默认容差不能直接满足 AOJ CGL_7_D 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result line_circle_intersections(RealPlane::Point a,
                                            RealPlane::Point b,
                                            RealPlane::Circle c,
                                            long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(c.r >= 0);
    Point v = b - a;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0) return {Kind::degenerate, {}};
        R z = G::cross(v, a - c.o);
        R d = c.r * c.r * q - z * z;
        if (d < 0) return {Kind::none, {}};
        Point h = c.o + G::perp(v) * (z / q);
        if (d == 0) return {Kind::one, {h}};
        Point w = v * (sqrtl(d) / q);
        return {Kind::two, {h - w, h + w}};
    }
    R len = G::norm(v);
    if (len == 0) return {Kind::degenerate, {}};
    Point h = line_projection(c.o, a, b);
    R d = G::norm(h - c.o), tol = eps * max(d, c.r);
    if (d > c.r + tol) return {Kind::none, {}};
    if (fabsl(d - c.r) <= tol) return {Kind::one, {h}};
    R t = sqrtl(max(R(0), (c.r - d) * (c.r + d)));
    Point w = v * (t / len);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；每条直线两端点不同，1≤q≤1000。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 1 页，1.1 节）、line_projection（第 3 页，1.2 节）、line_circle_intersections（第 6 页，1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle c;
    int q;
    cin >> c.o.x >> c.o.y >> c.r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_intersections(a, b, c, 0);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

1.5 浮点两圆交点

功能：求两圆的交点，区分相离或内含（无交点）、相切（一点）、相交（两点）以及两圆重合（无穷多）；半径为 0 的圆视为点。

核心思路：设 $v = o_2 - o_1$。交点连线与连心线的交点（垂足）为 $h = o_1 + v \cdot \frac{x}{|v|}$，其中 $x = \frac{|v|^2 + r_1^2 - r_2^2}{2|v|}$，两交点为 h 沿 $\mathrm{perp}(v)$ 方向偏移 $\pm\sqrt{r_1^2 - x^2}$。eps= 0 时全用平方量：$k = |v|^2 + r_1^2 - r_2^2$，判别式 $D = 4r_1^2|v|^2 - k^2$ 的符号决定交点个数。eps> 0 时先单独处理同心与点圆，再把圆心距与 $r_1 + r_2$、$|r_1 - r_2|$，以及 $|x|$ 与 r_1 按相对容差比较来分类。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_intersections(a,b,eps=1e-12L) 返回 Result；半径非负，eps 非负；以下相对阈值描述适用于 eps>0。两个完全相同的正半径圆返回 infinite；同位置两个点圆返回 one；同心不同半径返回 none。其余用距离和半径的相对阈值分类，点圆单独处理，接近相切可返回 one。输入有限且尺度适当，O(1) 时间空间；返回的有限交点仍受浮点构造误差影响。默认容差不能直接满足 AOJ CGL_7_E 的所有合法输入；近切时归并交点可能超出坐标误差限。eps=0 为显式无容差模式：直接计算平方判别式，不主动合并近切点；一般实数仍可能发生舍入误判、溢出或下溢。整数坐标绝对值 ≤10000、整数半径 0..10000，二进制 long double 尾数 ≥64 位且正常最近舍入、禁用 fast-math 时，中间整数精确。交点坐标仍为浮点近似；其他输入不继承这一保证。

```
RealPlane::Result
circle_intersections(RealPlane::Circle a, RealPlane::Circle b, long double eps = 1e-12L)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Kind = G::Kind;
    assert(eps >= 0);
    assert(a.r >= 0 && b.r >= 0);
    Point v = b.o - a.o;
    if (eps == 0)
    {
        R q = G::dot(v, v);
        if (q == 0)
        {
            if (a.r != b.r) return {Kind::none, {}};
            if (a.r == 0) return {Kind::one, {a.o}};
            return {Kind::infinite, {}};
        }
        R k = q + a.r * a.r - b.r * b.r;
        R d = 4 * a.r * a.r * q - k * k;
        if (d < 0) return {Kind::none, {}};
        Point h = a.o + v * (k / (2 * q));
        if (d == 0) return {Kind::one, {h}};
        Point w = G::perp(v) * (sqrtl(d) / (2 * q));
        return {Kind::two, {h - w, h + w}};
    }
```

默认正容差分支（接上页同一函数）：

```
    R d = G::norm(v);
    R tol = eps * max({d, a.r, b.r});
    // Infinite intersections require identical positive-radius circles.
    if (d == 0)
    {
        if (a.r != b.r) return {Kind::none, {}};
        if (a.r == 0) return {Kind::one, {a.o}};
        return {Kind::infinite, {}};
    }
    if (a.r == 0 && b.r == 0) return {Kind::none, {}};
    if (a.r == 0 || b.r == 0)
    {
        R radius = max(a.r, b.r);
        if (fabsl(d - radius) > tol) return {Kind::none, {}};
        return {Kind::one, {a.r == 0 ? a.o : b.o}};
    }
    if (d > a.r + b.r + tol || d < fabsl(a.r - b.r) - tol) return {Kind::none, {}};
    R x = (d * d + (a.r - b.r) * (a.r + b.r)) / (2 * d);
    Point h = a.o + v * (x / d);
    // Near-concentric circles must not be mistaken for internal tangency.
    if (fabsl(x) > a.r + tol) return {Kind::none, {}};
    if (fabsl(fabsl(x) - a.r) <= tol) return {Kind::one, {h}};
    R y = sqrtl(max(R(0), (a.r - x) * (a.r + x)));
    Point w = G::perp(v) * (y / d);
    return {Kind::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E —浮点接口无容差模式

题意：整数坐标绝对值 ≤10000、整数半径 1..10000，保证交点存在；两圆圆心不同。直接调用浮点接口并显式传 eps=0，不合并近切交点；仅在二进制 long double 尾数至少 64 位、正常最近舍入且禁用 fast-math 的平台采用此用法。此整数域的平方判别式精确，坐标仍为浮点构造；一般实数不继承精确分类保证。相切重复同一点，按普通 (x,y) 字典序输出，不能用 epsilon 排序；默认 eps>0 接口保持原近似语义。

所需模板：RealPlane（第 1 页，1.1 节）、circle_intersections（第 8 页，1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    static_assert(numeric_limits<long double>::radix == 2 &&
                  numeric_limits<long double>::digits >= 64);
#ifdef __FAST_MATH__
#error This example requires normal floating arithmetic
#endif
    if (fegetround() != FE_TONEAREST) return 1;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    auto ans = circle_intersections(a, b, 0);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

1.6 两圆交面积

问题描述：求两个圆公共部分（交集）的面积。

核心思路：圆心距 $d \geq r_1 + r_2$ 时不相交，面积为 0；$d \leq |r_1 - r_2|$ 时小圆被包含，面积为小圆面积。否则交集由两个弓形拼成：由余弦定理求出公共弦在每个圆中所对圆心角的一半 α，弓形面积为 $r^2(\alpha - \frac{1}{2}\sin 2\alpha)$，两者相加。$\alpha$ 很小时直接相减会严重相消，改用展开式 $\frac{2}{3}\alpha^3 - \frac{2}{15}\alpha^5 + \frac{4}{315}\alpha^7$。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_overlap_area(a,b) 返回 long double 面积，半径非负。分离或外切返回 0；内含时返回小圆面积，含点圆和同心圆。部分相交计算两个圆弓形面积，小角度使用展开式减轻相消；acos 入参截断至 [-1,1]。不使用交点分类容差；要求有限且尺度适当的输入。O(1) 时间空间，不把浮点输出视为精确面积。

```
RealPlane::R circle_overlap_area(RealPlane::Circle a, RealPlane::Circle b)
{
    using G = RealPlane;
    using R = G::R;
    assert(a.r >= 0 && b.r >= 0);
    R d = G::norm(a.o - b.o), pi = acosl(-1.L);
    if (d >= a.r + b.r) return 0;
    if (d <= fabsl(a.r - b.r)) return pi * min(a.r, b.r) * min(a.r, b.r);
    R difference = (a.r - b.r) * (a.r + b.r);
    R x = acosl(clamp((d * d + difference) / (2 * d * a.r), R(-1), R(1)));
    R y = acosl(clamp((d * d - difference) / (2 * d * b.r), R(-1), R(1)));
    auto segment = [](R angle)
    {
        if (angle < 1e-3L)
        {
            R z = angle * angle;
            return angle * z * (R(2) / 3 + z * (-R(2) / 15 + z * R(4) / 315));
        }
        return angle - sinl(2 * angle) / 2;
    };
    return a.r * a.r * segment(x) + b.r * b.r * segment(y);
}
```

模板题使用：AOJ CGL_7_I

题意：两个圆的公共面积，整数坐标绝对值 $\leq 10^4$、半径 $1..10^4$。覆盖相离、内含、同心与部分相交；输出面积的绝对误差须小于 1e-6。

所需模板：RealPlane（第 1 页，1.1 节）、circle_overlap_area（第 11 页，1.6 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_I

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Circle a, b;
    cin >> a.o.x >> a.o.y >> a.r >> b.o.x >> b.o.y >> b.r;
    cout << fixed << setprecision(12) << circle_overlap_area(a, b) << '\n';
}
```

相关应用：凸多边形裁剪：保留有向直线左侧；几何基础上的独立补充接口。

1.7 凸多边形裁剪：保留有向直线左侧

功能：用有向直线 $a \to b$ 切割凸多边形，返回多边形在直线左侧（含直线上）的部分，顶点保持原边界顺序。

核心思路：单条直线的 Sutherland–Hodgman 裁剪：令 $e = b - a$，依次考察每条边 $u \to v$，用 $s = e \times (u - a)$、$t = e \times (v - a)$ 判断两端在直线哪一侧。$s \geq 0$ 时保留顶点 u；两端严格异侧时再加入边与直线的交点 $u + (v - u)\frac{s}{s-t}$。端点恰在直线上时不求交，避免重复点。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_cut(p,a,b) 返回凸多边形与有向直线 a→b 左侧闭半平面的交。a≠b；输入按边界顺序，顺/逆时针均可，允许共线中间点；空集、点和线段也可。原输入不变，输出保留原方向，但可能有共线点或重复边界点，不能直接当作严格凸包传入 minimum_rectangle。

逐边保留左侧顶点，仅在两端严格异侧时加入交点，过顶点及沿边切割不作重复求交。O(n) 时间和输出空间。n 须能用 int 表示，输入及中间运算有限可表示。不使用 eps；任意近退化实数的符号受舍入影响，不保证精确拓扑。AOJ 整数范围内输入侧向叉积可精确表示，交点与面积仍舍入。

```
vector<RealPlane::Point> convex_cut(const vector<RealPlane::Point> &p,
    RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    assert(a.x != b.x || a.y != b.y);
    int n = int(p.size());
    vector<G::Point> q;
    auto e = b - a;
    for (int i = 0; i < n; i++)
    {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto s = G::cross(e, u - a);
        auto t = G::cross(e, v - a);
        if (s >= 0)
            q.push_back(u);
        if ((s < 0 && t > 0) || (s > 0 && t < 0))
            q.push_back(u + (v - u) * (s / (s - t)));
    }
    return q;
}
```

模板题使用：AOJ CGL_4_C Convex Cut

题意：n,q≤100，整数坐标绝对值 ≤10000，输入逆时针凸多边形可含共线点。每条查询均裁剪原多边形，输出左侧面积 12 位小数，要求绝对误差小于 1e-5。已核对官方题面与本地精确模型；AOJ 未登录且无已保存账号，尚无在线提交。

所需模板：RealPlane（第 1 页，1.1 节）、convex_cut（第 12 页，1.7 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_4_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, queries;
    cin >> n;
    vector<RealPlane::Point> p(n);
    for (auto &v : p)
        cin >> v.x >> v.y;
    cin >> queries;
    while (queries--)
    {
        RealPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto q = convex_cut(p, a, b);
        long double area = 0;
        long double error = 0;
        for (int i = 1; i + 1 < (int)q.size(); i++)
        {
            auto value = RealPlane::cross(q[i] - q[0], q[i + 1] - q[0]);
            auto y = value - error;
            auto next = area + y;
            error = (next - area) - y;
            area = next;
        }
        cout << fixed << setprecision(12) << abs(area) / 2 << '\n';
    }
}
```

相关应用：简单多边形面积重心；几何基础上的独立补充接口。

1.8 简单多边形面积重心

问题描述：求均匀密度简单多边形（可凹）的面积重心。

核心思路：以 p_0 为公共顶点把多边形剖成三角扇 (p_0, p_i, p_{i+1})。设 a, b 为相对 p_0 的向量，三角形有向面积的两倍为 $s = a \times b$，重心为 $(a+b)/3$；凹多边形中各三角形面积有正有负，恰好抵消多算的部分。累加 $\sum s$ 与 $\sum (a+b)s$，重心为 $p_0 + \frac{\sum (a+b)s}{3\sum s}$。三个累加量都用 Kahan 补偿求和（add）减小舍入误差。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_centroid(p) 返回 optional<RealPlane::Point>，表示均匀密度简单多边形的面积重心；可凹，顺/逆时针均可。少于三点或计算有向面积恰为 0 返回 nullopt，不把原点作为失败标记。不用于自交、多环带孔多边形，也不是顶点平均或边界线重心。

相对 p[0] 作三角扇，有向面积与一阶矩补偿求和；除以总有向面积恢复坐标。O(n) 时间、O(1) 额外空间，原输入不变。n 须能用 int 表示，输入与中间运算有限可表示；不使用 eps。实数近退化时面积符号和零判定仍可能舍入错误，巨大平移也限制最终坐标精度；long double 不保证比 double 精度高。

```
optional<RealPlane::Point> polygon_centroid(const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    int n = int(p.size());
    if (n < 3)
        return nullopt;
    array<R, 3> sum{};
    array<R, 3> error{};
    auto add = [&](int i, R v)
    {
        R y = v - error[i];
        R next = sum[i] + y;
        error[i] = (next - sum[i]) - y;
        sum[i] = next;
    };
    for (int i = 1; i + 1 < n; i++)
    {
        auto a = p[i] - p[0];
        auto b = p[i + 1] - p[0];
        R s = G::cross(a, b);
        add(0, s);
        add(1, (a.x + b.x) * s);
        add(2, (a.y + b.y) * s);
    }
    if (sum[0] == 0)
        return nullopt;
    G::Point offset{sum[1] / sum[0] / 3, sum[2] / sum[0] / 3};
    return p[0] + offset;
}
```

相关应用：三角形内心与内切圆，非完整二维几何来源覆盖

1.9 三角形内切圆：边长加权内心

问题描述：给定三角形的三个顶点，求其内切圆的圆心（内心）和半径。

核心思路：内心是三个顶点以各自对边长为权的加权平均。相对 a 平移得 $u=b-a$，$v=c-a$，三顶点的对边长分别为 $|v-u|$、$|v|$、$|u|$，于是内心为 $a+\frac{|v|u+|u|v}{|u|+|v|+|v-u|}$；半径为面积除以半周长，即 $|u\times v|$ 除以周长。代码先用原坐标求叉积判断退化，再把 u,v 除以最大分量 `scale` 后求边长，最后乘回，避免溢出。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：triangle_incircle(a,b,c) 返回 optional<RealPlane::Circle>，圆心为内心、半径为面积除以半周长。顺逆时针均可，不改变输入；计算叉积恰为 0 或三点全重合返回 nullopt，不把原点作为失败标志。用对边长加权顶点，避免三角函数；O(1) 时间和空间。

先相对 a 平移，边向量按最大分量归一化再求边长；面积叉积在归一化前计算，保留 AOJ 整数域内精确零判定。要求输入、相减、叉积及输出等中间值有限可表示且不发生影响结果的下溢；任意近共线实数的零判定仍可能舍入错误，不是精确拓扑接口。AOJ 整数坐标绝对值 <=10000、三点不共线；叉积精确，圆心半径仍为浮点近似。long double 不保证比 double 精度高。

```
optional<RealPlane::Circle> triangle_incircle(RealPlane::Point a,
                                              RealPlane::Point b,
                                              RealPlane::Point c)
{
    using G = RealPlane;
    using R = G::R;
    auto u = b - a, v = c - a;
    R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
    if (scale == 0)
        return nullopt;
    R area = fabsl(G::cross(u, v));
    if (area == 0)
        return nullopt;
    u = u / scale;
    v = v / scale;
    R x = G::norm(v - u), y = G::norm(v), z = G::norm(u);
    R sum = x + y + z;
    auto o = a + (u * (y / sum) + v * (z / sum)) * scale;
    return G::Circle{o, area / scale / sum};
}
```

模板题使用：AOJ CGL_7_B 三角形内切圆

题意：输入三个非共线整点，坐标绝对值不超过 10000，输出内心坐标与半径；要求误差小于 1e-6。官方题面已核对，完整协议本地验证，尚无本实现在线记录。

所需模板：RealPlane（第 1 页，1.1 节）、triangle_incircle（第 15 页，1.9 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_B&lang=en

```
int main()
{
    RealPlane::Point a, b, c;
    cin >> a.x >> a.y >> b.x >> b.y >> c.x >> c.y;
    auto circle = triangle_incircle(a, b, c);
    if (circle)
        cout << fixed << setprecision(12) << circle->o.x << ' '
             << circle->o.y << ' ' << circle->r << '\n';
}
```

1.10 有限二进制浮点极角精确排序

功能：浮点点的极角比较器：从正 x 轴起按逆时针极角排序，同方向按长度递增，原点排在最前；对已存入的浮点数比较结果精确。

核心思路：先按半平面分组（`half`：上半平面及正 x 轴在前），同组内看叉积 $a_x b_y - a_y b_x$ 的符号。为避免浮点叉积舍入，`product` 用 `frexp` 把每个因子拆成 64 位整数尾数和指数，尾数相乘得 128 位整数并左移规格化，于是乘积可按（符号，指数，规格化尾数）精确比较大小；`cross_sign` 据此比较两个乘积。叉积为 0（同方向）时比较非零坐标分量的绝对值。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：RealPolarLess{}(a,b) 沿用 RealPlane::Point 存储，输入为实际已存入的有限二进制 long double，尾数 ≤64 位且 uint128 可用；禁 fast-math 及 FTZ/DAZ。原点最前，正 x 轴起逆时针 [0,2π)，同向按长度递增，同点及 ±0 等价，满足严格弱序。用 frexp/ldexp 拆尾数并以 uint128 精确比较乘积，不做浮点叉乘、不用 epsilon。每次 O(1)，无分配。十进制输入先舍入；不修复调用前的坐标差，不替代其他浮点几何谓词。product/cross_sign 是有限输入下的内部辅助。

```
struct RealPolarLess
{
    using R = RealPlane::R;
    using P = RealPlane::Point;
    using U = __uint128_t;
    static_assert(numeric_limits<R>::radix == 2 && numeric_limits<R>::digits <= 64);

    struct Product
    {
        int sign;
        long long exponent;
        U value;
    };

    static Product product(R a, R b)
    {
        if (a == 0 || b == 0) return {0, 0, 0};
        int x, y;
        constexpr int p = numeric_limits<R>::digits;
        uint64_t u = ldexp(frexp(abs(a), &x), p);
        uint64_t v = ldexp(frexp(abs(b), &y), p);
        U m = U(u) * v;
        uint64_t hi = m >> 64;
        int bits = hi ? 64 + bit_width(hi) : bit_width(uint64_t(m));
        return {signbit(a) == signbit(b) ? 1 : -1,
                (long long)x + y - 2 * p + bits, m << (128 - bits)};
    }

    static int cross_sign(P a, P b)
    {
        Product u = product(a.x, b.y), v = product(a.y, b.x);
        if (u.sign != v.sign) return u.sign < v.sign ? -1 : 1;
        if (!u.sign) return 0;
        int c = u.exponent != v.exponent ? (u.exponent < v.exponent ? -1 : 1)
              : u.value != v.value ? (u.value < v.value ? -1 : 1) : 0;
        return u.sign * c;
    }
```

接上页同一结构体：极角比较接口。

```
    bool operator()(P a, P b) const
    {
        assert(isfinite(a.x) && isfinite(a.y) && isfinite(b.x) && isfinite(b.y));
        bool az = a.x == 0 && a.y == 0, bz = b.x == 0 && b.y == 0;
        if (az || bz) return az && !bz;
        auto half = [](P p) { return p.y < 0 || (p.y == 0 && p.x < 0); };
        if (half(a) != half(b)) return half(a) < half(b);
        int c = cross_sign(a, b);
        if (c) return c > 0;
        return a.x != 0 ? abs(a.x) < abs(b.x) : abs(a.y) < abs(b.y);
    }
};
```

1.11 整数点向量与线段判定

功能：整数坐标的二维几何基础：点类型及字典序比较、叉积、点积、距离平方，判断点是否在线段上、两条闭线段是否相交，以及极角比较器 `PolarLess`。

核心思路：所有运算先把坐标差扩展到 `__int128_t` 再相乘，全程精确，无需 eps。`cross(a,b,c)` 为 $(b-a)\times(c-a)$，正值表示 c 在 ab 左侧。`on_segment` 要求三点共线且 $(a-p)\cdot(b-p)\le 0$。`intersect` 先处理某端点与另一线段共线并落在其上的接触情形，否则要求两线段的端点分别严格位于对方直线两侧。`PolarLess` 先按半平面分组，组内用叉积比较极角，同向比长度。

接口契约：坐标绝对值不超过 10^{12}，使用 long long 存点、___int128_t 计算叉积、点积与距离平方。Point 按 x、y 字典序比较；cross(a,b,c) 为 ab 与 ac 的叉积，dot 为点积。on_segment 包含端点及退化线段；intersect 判断闭线段有交点。PolarLess 从正 x 轴按 [0,2π) 逆时针极角排序，同向按长度平方递增；零向量排在最前。同点等价，比较器满足严格弱序。按其他起点排序应整体循环移动，LC 的 (-π,π] 顺序仅将 y<0 后缀移到前面。全部单次运算 O(1)，不使用 epsilon。

```cpp
struct IntegerPlane
{
    using ll = long long;
    using I = __int128_t;

    // Contract: |coordinate|<=1e12. Differences/products widened before arithmetic.
    struct Point
    {
        ll x = 0, y = 0;

        bool operator<(Point b) const { return tie(x, y) < tie(b.x, b.y); }

        bool operator==(Point b) const { return x == b.x && y == b.y; }
    };

    static I cross(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.y) - a.y) - (I(b.y) - a.y) * (I(c.x) - a.x);
    }

    static I dot(Point a, Point b, Point c)
    {
        return (I(b.x) - a.x) * (I(c.x) - a.x) + (I(b.y) - a.y) * (I(c.y) - a.y);
    }

    static I dist2(Point a, Point b) { return dot(a, b, b); }

    static int sign(I x) { return (x > 0) - (x < 0); }

    static bool on_segment(Point p, Point a, Point b)
    {
        return cross(a, b, p) == 0 && dot(p, a, b) <= 0;
    }

    static bool intersect(Point a, Point b, Point c, Point d)
    {
        int x = sign(cross(a, b, c)), y = sign(cross(a, b, d));
        int z = sign(cross(c, d, a)), w = sign(cross(c, d, b));
        if (!x && on_segment(c, a, b)) return true;
        if (!y && on_segment(d, a, b)) return true;
        if (!z && on_segment(a, c, d)) return true;
        if (!w && on_segment(b, c, d)) return true;
        return x * y < 0 && z * w < 0;
    }
```

极角比较器（接上页同一结构体）：

```
    struct PolarLess
    {
        static int half(Point p) { return p.y < 0 || (p.y == 0 && p.x < 0); }

        bool operator()(Point a, Point b) const
        {
            if (half(a) != half(b)) return half(a) < half(b);
            I c = cross({0, 0}, a, b);
            if (c) return c > 0;
            return dist2({0, 0}, a) < dist2({0, 0}, b);
        }
    };
};
```

模板题使用：Library Checker sort_points_by_argument

题意：将最多 20 万个整数点按 $(-\pi,\pi]$ 的极角排序，坐标绝对值 $\le 10^9$，约定原点角度为 0，同角顺序任意。PolarLess 从正 x 轴按 $[0,2\pi)$ 排序，再把 y<0 的后缀整体移到前面；负 x 轴留在最后，不要把它和负 y 半平面一起前移。仅调用整数点与极角比较器。

所需模板：IntegerPlane（第 19 页，1.11 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/sort_points_by_argument

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    sort(p.begin(), p.end(), IntegerPlane::PolarLess{});
    auto first = find_if(p.begin(), p.end(), [](auto v) { return v.y < 0; });
    rotate(p.begin(), first, p.end());
    for (auto v : p) cout << v.x << ' ' << v.y << '\n';
}
```

1.12 整数多边形有向面积

功能：求按边界顺序给出的整数多边形有向面积的两倍（逆时针为正）。

核心思路：鞋带公式：$2S = \sum_i p_i \times p_{i+1}$（下标循环）。每一项是原点与一条边构成的三角形的有向面积两倍，多边形外的部分正负抵消。乘积在 `__int128_t` 中精确计算。

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_area2(p) 返回 ___int128_t 的有向面积两倍；逆时针为正，反向为负。输入按边界顺序排列，隐式闭合；空集或不足三个点返回 0。单个多边形点数不超过 10^5、坐标绝对值不超过 10^{12}，确保求和在有符号 128 位内。时间 O(n)，额外空间 O(1)。

```
IntegerPlane::I polygon_area2(const vector<IntegerPlane::Point> &p)
{
    using G = IntegerPlane;
    using I = G::I;
    I ans = 0;
    int n = (int)p.size();
    for (int i = 0; i < n; i++)
        ans += I(p[i].x) * p[(i + 1) % n].y - I(p[i].y) * p[(i + 1) % n].x;
    return ans;
}
```

模板题使用：AOJ CGL_3_A

题意：给出逆时针简单多边形，3 至 100 点，整数坐标绝对值不超过 10000，输出面积并保留一位小数。无需凸包，凹多边形也适用。接口返回有向二倍面积，逆时针为正；取绝对值后按奇偶精确打印.0 或.5。本题范围允许将半面积转为 long long；一般 int128 结果不可任意缩窄。

所需模板：IntegerPlane（第 19 页，1.11 节）、polygon_area2（第 22 页，1.12 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_A

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    auto s = polygon_area2(p);
    if (s < 0) s = -s;
    cout << (long long)(s / 2) << (s % 2 ? ".5\n" : ".0\n");
}
```

1.13 点在整数简单多边形内

问题描述：判断点 q 在简单多边形（可凹，顺逆时针均可）的外部、边界上还是内部。

核心思路：计算绕数。逐边 $a \to b$：若 q 在边上直接返回边界。否则用半开纵坐标区间判断边是否穿过过 q 的水平线：向上穿过（$a_y \le q_y < b_y$）且 q 在边左侧时绕数加一，向下穿过（$b_y \le q_y < a_y$）且 q 在边右侧时减一。半开区间保证顶点恰在水平线上时只计一次；绕数非零即在内部。

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_contains(p,q) 返回 0 外部、1 边界、2 内部，输入简单多边形，可顺时针或逆时针，不重复首点。空多边形返回 0；单点与线段按闭集判断边界。绕数算法先判线段，再按半开纵坐标区间累计，避免顶点重复计数。坐标范围同 IntegerPlane，O(n) 时间、O(1) 空间。

```
int polygon_contains(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using Point = G::Point;
    using I = G::I;
    int winding = 0, n = (int)p.size();
    for (int i = 0; i < n; i++)
    {
        Point a = p[i], b = p[(i + 1) % n];
        if (G::on_segment(q, a, b)) return 1;
        I c = G::cross(a, b, q);
        if (a.y <= q.y && b.y > q.y && c > 0) ++winding;
        if (a.y > q.y && b.y <= q.y && c < 0) --winding;
    }
    return winding ? 2 : 0;
}
```

模板题使用：AOJ CGL_3_C

题意：给定简单多边形和查询点，逐个输出 0 外部、1 边界、2 内部；允许凹多边形，按题目给出的逆时针边界顺序读入，不要求重建凸包。n 至 100、q 至 1000，坐标绝对值至 10000；核心也接受反向边界。

所需模板：IntegerPlane（第 19 页，1.11 节）、polygon_contains（第 23 页，1.13 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_3_C

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    vector<IntegerPlane::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y;
    cin >> q;
    while (q--)
    {
        IntegerPlane::Point v;
        cin >> v.x >> v.y;
        cout << polygon_contains(p, v) << '\n';
    }
}
```

1.14 整数输入直线与圆交点

功能：整数输入的直线与圆求交：直线由两整点给出，圆心为整点、半径为整数，返回交点个数类别与交点坐标。

核心思路：先在 `__int128_t` 中精确分类：$v = b - a$，$c = v \times (a - o)$，圆心到直线距离为 $|c|/|v|$，判别式 $D = r^2|v|^2 - c^2$ 的符号精确决定相离、相切或相交。之后才转成 `long double` 构造坐标：垂足 $h = o + \mathrm{perp}(v)\, c/|v|^2$，交点为 $h \mp v\sqrt{D}/|v|^2$。

代码依赖：IntegerPlane（第 19 页，1.11 节）、RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：line_circle_i64(a,b,o,r)：a、b 定直线，o 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。a=b 返回 degenerate；相离 none、相切 one、相交 two。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result line_circle_i64(IntegerPlane::Point a,
                                  IntegerPlane::Point b,
                                  IntegerPlane::Point o,
                                  long long r)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b, o})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    if (q == 0) return {K::degenerate, {}};
    I c = x * (I(a.y) - o.y) - y * (I(a.x) - o.x);
    I d = I(r) * r * q - c * c;
    if (d < 0) return {K::none, {}};
    RealPlane::Point v{R(x), R(y)}, center{R(o.x), R(o.y)};
    auto h = center + RealPlane::perp(v) * (R(c) / R(q));
    if (d == 0) return {K::one, {h}};
    auto w = v * (sqrtl(R(d)) / R(q));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_D

题意：整数圆心/半径及直线，坐标绝对值 $\leq 10^4$、半径 $1..10^4$，q≤1000。题目保证直线非退化且至少一个交点；精确分类后构造坐标，按 x/y 字典序输出，相切点复制一次。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 19 页，1.11 节）、RealPlane（第 1 页，1.1 节）、line_circle_i64（第 24 页，1.14 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point o;
    long long r;
    int q;
    cin >> o.x >> o.y >> r >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        IntegerPlane::Point a, b;
        cin >> a.x >> a.y >> b.x >> b.y;
        auto ans = line_circle_i64(a, b, o, r);
        assert(ans.p.size() == 1 || ans.p.size() == 2);
        if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
        if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
            swap(ans.p[0], ans.p[1]);
        cout << ans.p[0].x << ' ' << ans.p[0].y << ' ' << ans.p[1].x << ' '
             << ans.p[1].y << '\n';
    }
}
```

1.15 整数输入两圆交点

功能：整数输入的两圆求交：圆心为整点、半径为整数，返回交点个数类别与交点坐标。

核心思路：设 $v = b - a$，$q = |v|^2$。同心时按半径是否相等、是否为 0 区分无交点、一点和无穷多。否则在 `__int128_t` 中计算 $k = q + r^2 - s^2$ 与判别式 $D = 4r^2q - k^2$，其符号精确决定交点个数；再用浮点构造：连心线上的垂足 $h = a + v\,k/(2q)$，两交点为 $h \mp \mathrm{perp}(v)\sqrt{D}/(2q)$。

代码依赖：IntegerPlane（第 19 页，1.11 节）、RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_intersections_i64(a,r,b,s)：a、b 为圆心。依赖 IntegerPlane::Point 及 RealPlane::Result。所有输入坐标绝对值 $\le 10^9$，半径在 $0..10^9$；先用 int128 精确判定交点数，再用 long double 构造坐标。相离或严格内含返回 none、相切 one、相交 two；同心等正半径返回 infinite，同心零半径返回 one。结果 p 不排序，one 只含一个点，none/infinite/degenerate 为空。零半径按点圆处理。O(1) 时间和空间。精确的是分类，坐标仍有舍入误差；在 10^9 尺度不承诺绝对 1e-6。AOJ 整数范围 10^4 内的驱动按字典序输出，相切点重复一次。本地高精度与消毒器测试不等于线上 AC。

```
RealPlane::Result circle_intersections_i64(IntegerPlane::Point a,
                                           long long r,
                                           IntegerPlane::Point b,
                                           long long s)
{
    using I = __int128_t;
    using R = RealPlane::R;
    using K = RealPlane::Kind;
    for (auto p : {a, b})
        assert(-1000000000 <= p.x && p.x <= 1000000000 && -1000000000 <= p.y &&
               p.y <= 1000000000);
    assert(0 <= r && r <= 1000000000 && 0 <= s && s <= 1000000000);
    I x = I(b.x) - a.x, y = I(b.y) - a.y;
    I q = x * x + y * y;
    RealPlane::Point center{R(a.x), R(a.y)}, v{R(x), R(y)};
    if (q == 0)
    {
        if (r != s) return {K::none, {}};
        if (r == 0) return {K::one, {center}};
        return {K::infinite, {}};
    }
    I k = q + I(r) * r - I(s) * s;
    I d = 4 * I(r) * r * q - k * k;
    if (d < 0) return {K::none, {}};
    auto h = center + v * (R(k) / (2 * R(q)));
    if (d == 0) return {K::one, {h}};
    auto w = RealPlane::perp(v) * (sqrtl(R(d)) / (2 * R(q)));
    return {K::two, {h - w, h + w}};
}
```

模板题使用：AOJ CGL_7_E

题意：整数圆心/半径，坐标绝对值 $\leq 10^4$、半径 $1..10^4$；不同圆心且至少一个交点。按 x/y 字典序输出，相切点复制一次；不使用 eps 排序。绝对误差要求小于 1e-6。

所需模板：IntegerPlane（第 19 页，1.11 节）、RealPlane（第 1 页，1.1 节）、circle_intersections_i64（第 26 页，1.15 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_E

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    IntegerPlane::Point a, b;
    long long r, s;
    cin >> a.x >> a.y >> r >> b.x >> b.y >> s;
    auto ans = circle_intersections_i64(a, r, b, s);
    assert(ans.p.size() == 1 || ans.p.size() == 2);
    if (ans.p.size() == 1) ans.p.push_back(ans.p[0]);
    if (tie(ans.p[1].x, ans.p[1].y) < tie(ans.p[0].x, ans.p[0].y))
        swap(ans.p[0], ans.p[1]);
    cout << fixed << setprecision(12) << ans.p[0].x << ' ' << ans.p[0].y << ' '
         << ans.p[1].x << ' ' << ans.p[1].y << '\n';
}
```

1.16 圆与简单多边形相交面积

问题描述：求圆与简单多边形（可凹，顺逆时针均可）公共部分的面积。

核心思路：把圆心平移到原点，对多边形每条有向边 $a \to b$ 计算“原点与该边所成三角形和圆的交”的有向面积，求和取绝对值即为答案（与鞋带公式同理，凹处正负抵消）。`edge` 先求边与圆的交点，把边切成至多三段；某段中点在圆内则整段在圆内，贡献三角形面积 $\frac{1}{2}\,p \times q$，否则贡献扇形面积 $\frac{1}{2}r^2\theta$，有向角 θ 由 `atan2l` 求出。各边贡献用 Kahan 补偿求和。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：area(points,circle) 返回非负相交面积；多边形须简单，允许凹多边形、顺时针/逆时针和相邻重复顶点。少于三点或半径零返回零。edge(a,b,r) 返回圆心在原点时有向边 a 到 b 的面积贡献，r 必须非负。按交点分段，内部用三角形，外部用有向扇形；补偿求和，O(n) 时间、O(1) 额外空间。依赖浮点几何的 Point/Circle 和向量运算。输入须有限、尺度适当，平移到圆心后作差、乘积和面积均须可表示；极小交叠及接近相切时仍有浮点误差。

```
struct CirclePolygon
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    // Signed contribution of edge a -> b; circle centered at the origin.
    static R edge(Point a, Point b, R r)
    {
        if (r == 0) return 0;
        Point v = b - a;
        R length = G::norm(v);
        if (length == 0) return 0;
        Point u = v / length;
        R h = G::cross(u, a);
        Point cuts[4] = {a};
        int count = 1;
        if (fabsl(h) <= r)
        {
            R delta = sqrtl(max(R(0), (r - fabsl(h)) * (r + fabsl(h))));
            R t = -G::dot(a, u);
            Point foot = G::perp(u) * h;
            for (R s : {-delta, delta})
                if (t + s > 0 && t + s < length) cuts[count++] = foot + u * s;
        }
        cuts[count++] = b;
```

分段面积与逐边累加（接上页同一结构体与函数）：

```
        R answer = 0;
        for (int i = 1; i < count; i++)
        {
            Point p = cuts[i - 1], q = cuts[i];
            R cross = G::cross(p, q);
            if (G::norm((p + q) / 2) <= r)
                answer += cross / 2;
            else
                answer += r * r * atan2l(cross, G::dot(p, q)) / 2;
        }
        return answer;
    }

    // Simple polygon, either orientation; duplicate consecutive vertices allowed.
    static R area(const vector<Point> &p, Circle c)
    {
        assert(c.r >= 0);
        if (p.size() < 3 || c.r == 0) return 0;
        R answer = 0, error = 0;
        for (int i = 0; i < (int)p.size(); i++)
        {
            R value = edge(p[i] - c.o, p[(i + 1) % p.size()] - c.o, c.r);
            R y = value - error;
            R sum = answer + y;
            error = (sum - answer) - y;
            answer = sum;
        }
        return fabsl(answer);
    }
};
```

模板题使用：AOJ CGL_7_H

题意：原点圆与逆时针简单多边形的公共面积，允许凹多边形。n 为 3..100，整数坐标绝对值 ≤100，半径 1..100。直接传入顶点序列，不取凸包；绝对误差小于 1e-5。

所需模板：RealPlane（第 1 页，1.1 节）、CirclePolygon（第 28 页，1.16 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_H

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long double r;
    cin >> n >> r;
    vector<RealPlane::Point> p(n);
    for (auto &a : p) cin >> a.x >> a.y;
    cout << fixed << setprecision(12) << CirclePolygon::area(p, {{0, 0}, r}) << '\n';
}
```

1.17 两圆公切线与过点切线

问题描述：求两圆的全部公切线（外公切线与内公切线），或过一点作圆的切线；返回每条切线在两圆上的切点和单位法向量，两圆重合时报告无穷多条。

核心思路：设 u 为连心线单位向量，d 为圆心距。切线的单位法向 n 满足 $n \cdot u = c$：外公切线（`side`$= 1$）$c = (r_1 - r_2)/d$，内公切线（`side`$= -1$）$c = (r_1 + r_2)/d$；$|c| > 1$ 时该类切线不存在。两条切线的法向为 $n = cu \pm \sqrt{1 - c^2}\,\mathrm{perp}(u)$，切点为 $o_1 + r_1 n$ 与 $o_2 + s\,r_2 n$（s 为 `side`）。$|c|$ 接近 1 时两圆相切，只输出一条。点视为半径 0 的圆，`from_point` 直接复用；两点时返回过两点的直线。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(a,b,eps) 返回 Result，包含 infinite 和 lines；infinite 为真表示无穷多条，否则 lines 给出全部切线。每条 Tangent 保存切点 a、b，单位法向量 normal，以及 side。a、b 对应输入两圆，直线方程 dot(normal,x-a)=0；切点相同时仍有效。side=1 外公切线、-1 内公切线、0 两个不同点。半径零表示点，from_point(p,c,eps) 等价于第一圆取点，圆上切点读 b。输入有限，坐标及半径绝对值 ≤1e100，半径非负。eps 默认 1e-12，用于相对圆心距的近相切判断；近相切可能合并为一条，浮点坐标仍有舍入误差。严格同心等半径时 infinite=true，包括同一点；同心不等半径返回空。O(1)；依赖浮点几何的 Point/Circle、点运算及 norm/perp。

```
struct CircleTangents
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    struct Tangent
    {
        Point a, b, normal;
        int side; // 1 outer, -1 inner, 0 two distinct point circles.
    };

    struct Result
    {
        bool infinite = false;
        vector<Tangent> lines;
    };

    // Line: dot(normal, x - a) = 0. Radius zero represents a point.
    static Result solve(Circle a, Circle b, R eps = 1e-12L)
    {
        assert(a.r >= 0 && b.r >= 0 && eps > 0 && eps < 1);
        Point v = b.o - a.o;
        R d = G::norm(v);
        if (d == 0) return {a.r == b.r, {}};
        Point u = v / d;
        if (a.r == 0 && b.r == 0) return {false, {{a.o, b.o, G::perp(u), 0}}};
```

内外公切线枚举（接上页同一函数）：

```
        Result answer;
        for (int side : {1, -1})
        {
            if (side == -1 && (a.r == 0 || b.r == 0)) continue;
            R gap = a.r - side * b.r;
            if (fabsl(gap) > d * (1 + eps)) continue;
            bool touch = fabsl(gap) >= d * (1 - eps);
            R c = touch ? (gap < 0 ? -1 : 1) : gap / d;
            R s = sqrtl(max(R(0), (1 - c) * (1 + c)));
            for (int sign : {-1, 1})
            {
                Point normal = u * c + G::perp(u) * (sign * s);
                Point p = a.o + normal * a.r;
                Point q = b.o + normal * (side * b.r);
                answer.lines.push_back({p, q, normal, side});
                if (touch) break;
            }
        }
        return answer;
    }

    static Result from_point(Point p, Circle c, R eps = 1e-12L)
    {
        return solve({p, 0}, c, eps);
    }
};
```

模板题使用：AOJ CGL_7_F

题意：过圆外整数点作切线，输出圆上的两个切点。坐标绝对值 ≤1000，半径 1..1000，保证点严格在圆外。from_point 把点作为第一圆，圆上切点在 line.b；按 x/y 字典序输出，每行一个点，绝对误差小于 1e-5。

所需模板：RealPlane（第 1 页，1.1 节）、CircleTangents（第 30 页，1.17 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_7_F

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    RealPlane::Point p;
    RealPlane::Circle c;
    cin >> p.x >> p.y >> c.o.x >> c.o.y >> c.r;
    auto ans = CircleTangents::from_point(p, c);
    vector<RealPlane::Point> points;
    for (auto line : ans.lines) points.push_back(line.b);
    sort(points.begin(),
         points.end(),
         [](auto a, auto b) { return tie(a.x, a.y) < tie(b.x, b.y); });
    cout << fixed << setprecision(12);
    for (auto a : points) cout << a.x << ' ' << a.y << '\n';
}
```

相关应用：任意简单整数多边形面积并；符号化右侧极限、共线边编号去重和有理参数排序，浮点面积输出。

1.18 一般多边形面积并：整数精确判定

问题描述：给定若干整数顶点的简单多边形（可凹，彼此可交叉、重叠），求它们并集的面积。

核心思路：并集边界由各多边形的边中不被其他多边形覆盖的部分组成，面积等于对这些部分用鞋带公式求和。先把每个多边形统一成逆时针。对每条边 pq，枚举其他多边形的每条边，求其穿过直线 pq 处的参数 t（分数精确表示，截到 $[0,1]$），按穿越方向记 ±1 覆盖事件；排序后扫描，覆盖数为 0 的参数区间按长度比例计入 $\frac{1}{2}(p-o)\times(q-o)$。两边共线同向时只让编号小的多边形计入，避免重合边界算两次。

思路中的情况	代码中的处理	行
规范化	删去连续重复点和末尾重复首点，零面积多边形丢弃，顺时针的翻转为逆时针	36
选局部原点	包围盒相交的多边形用并查集合并，同一块共用一个原点 o 以减小浮点误差	62
别的边穿过直线 pq	交点参数为 $\frac{(d-c)\times(p-c)}{y-x}$，从右向左穿越记 -1（离开），反之记 $+1$	91
共线重合边	同向重叠部分只在编号较小的多边形中计一次：对编号大者把重叠区间标为已覆盖	92
扫描参数	覆盖数为 0 的区间对应并集边界，按比例累加面积（Kahan 补偿）	118

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：polygon_union_area(polygons) 求任意简单多边形的面积并，允许凹多边形、不同多边形边界交叉/重合以及并集孔洞；每个输入多边形本身不含洞、不自交。整数顶点为 IntegerPlane::Point，坐标绝对值 $\le 10^9$（比基础点类型约定更严格）。自动转逆时针，删除连续重复点及末尾重复首点；零面积退化输入忽略，但不负责检查自交。

共 V 个顶点，时间 $O(V^2 \log V)$、空间 $O(V)$，下标/覆盖数须可用 int 表示。128 位叉积与有理参数排序精确处理拓扑，最终面积 long double 近似；不扰动顶点、不用 eps 排序。包围盒相交的多边形共用局部原点并补偿求和，仍须按题目精度检验最终面积，不声称浮点累加无条件精确。

```
long double polygon_union_area(vector<vector<IntegerPlane::Point>> polygons)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    using R = long double;
    struct Fraction
    {
        I num, den;
        Fraction(I x = 0, I y = 1) : num(x), den(y)
        {
            assert(den != 0);
            if (den < 0)
            {
                num = -num;
                den = -den;
            }
        }
        bool operator<(const Fraction &b) const
        {
            return num * b.den < b.num * den;
        }
    };
    vector<vector<P>> a;
    for (auto p : polygons)
    {
        for (auto v : p)
            assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                   -1000000000 <= v.y && v.y <= 1000000000);
        p.erase(unique(p.begin(), p.end()), p.end());
        if (p.size() > 1 && p.front() == p.back()) p.pop_back();
        I area = 0;
        int m = p.size();
        for (int i = 0; i < m; i++) area += G::cross({0, 0}, p[i], p[(i + 1) % m]);
        if (area == 0) continue;
        if (area < 0) reverse(p.begin(), p.end());
        a.push_back(move(p));
    }
```

局部原点与边事件初始化（接上页函数）：

```
    int n = a.size();
    vector<array<long long, 4>> box(n);
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
    {
        auto p = a[i][0];
        box[i] = {p.x, p.y, p.x, p.y};
        for (auto v : a[i])
        {
            box[i][0] = min(box[i][0], v.x);
            box[i][1] = min(box[i][1], v.y);
            box[i][2] = max(box[i][2], v.x);
            box[i][3] = max(box[i][3], v.y);
        }
        for (int j = 0; j < i; j++)
            if (max(box[i][0], box[j][0]) <= min(box[i][2], box[j][2]) &&
                max(box[i][1], box[j][1]) <= min(box[i][3], box[j][3]))
                parent[find(find, i)] = find(find, j);
    }
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        int m = a[i].size();
        P origin = a[find(find, i)][0];
        for (int k = 0; k < m; k++)
        {
            P p = a[i][k];
            P q = a[i][(k + 1) % m];
            vector<pair<Fraction, int>> events;
            auto add = [&](Fraction t, int delta)
            {
                t = max(Fraction(0), min(Fraction(1), t));
                events.push_back({t, delta});
            };
```

右侧覆盖与共线边去重（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            int size = a[j].size();
            for (int h = 0; h < size; h++)
            {
                P c = a[j][h];
                P d = a[j][(h + 1) % size];
                I x = G::cross(p, q, c);
                I y = G::cross(p, q, d);
                if ((x >= 0) != (y >= 0))
                    add(Fraction(G::cross(c, d, p), y - x), x < 0 ? -1 : 1);
                if (x == 0 && y == 0 && j < i)
                {
                    auto project = [&](P v)
                    {
                        if (p.x != q.x) return Fraction(I(v.x) - p.x, I(q.x) - p.x);
                        return Fraction(I(v.y) - p.y, I(q.y) - p.y);
                    };
                    auto l = project(c);
                    auto r = project(d);
                    if (l < r)
                    {
                        add(l, 1);
                        add(r, -1);
                    }
                }
            }
        }
```

有理参数排序及未覆盖段积分（接上页函数）：

```
            add(Fraction(1), 0);
            sort(events.begin(), events.end(), [](const auto &x, const auto &y)
            {
                return x.first < y.first;
            });
            int cover = 0;
            Fraction last;
            for (auto [t, delta] : events)
            {
                if (cover == 0)
                {
                    I num = t.num * last.den - last.num * t.den;
                    I den = t.den * last.den;
                    R value = R(G::cross(origin, p, q)) * (R(num) / R(den)) / 2;
                    R y = value - error;
                    R next = answer + y;
                    error = (next - answer) - y;
                    answer = next;
                }
                cover += delta;
                last = t;
            }
        }
    }
    return answer;
}
```

精确范围 令坐标界 M=10 的 9 次方。差向量分量至多 2M，叉积、交点参数分子及分母绝对值至多 8M 平方。分式比较的单个交叉乘积至多 64M 四次方，小于 128 位有符号上界；裁到 [0,1] 后的参数差也安全。不能直接沿用 IntegerPlane 允许的更大坐标界。

为何看边的右侧 每个多边形先转成逆时针，当前边的左侧已在并集中。只有其他多边形占据右侧时，该边才成为并集内部，不能贡献边界。把测试线向右侧平移无穷小量，原线上顶点视作在线上方：cross 大于等于 0 对比小于 0，等价于符号化极限，没有实际扰动坐标。向下穿越时覆盖加 1，向上穿越时减 1。凹多边形会产生多个区间，仍由相同事件累计处理。

共享边与端点 两条共线反向边的内部位于两侧，右侧测试已将其剔除；同向共线边仅编号较小的多边形保留，避免重复积分。所有事件参数精确裁到 [0,1]，同参数事件的任意顺序只影响零长度段，不影响面积。退化顶点、相切和部分重叠边均不需要随机扰动。

积分与孔洞 对边 p 到 q 的未覆盖参数段 [l,u]，贡献是 cross(p-origin,q-origin) 乘 (u-l)/2。按逆时针原边方向累加，孔洞自动带负方向，不对单边取绝对值。包围盒相交关系的连通块包含真实并集连通块，按该分组平移不改变闭合积分。最终转 long double 和补偿求和仍有误差；输入内含洞或自交轮廓不在本接口范围。

独立验证 Fraction 精确枚举所有线段交点横坐标，分竖直条带合并各多边形截面，以线性上下包络的梯形公式求真值。包含凹多边形、重复/共线/反向边、孔洞、极值坐标、刚体变换和 1000 三角形压力例。线上状态见验证记录。

模板题使用：P13338 三角形面积并加强版

题意：n≤1000 个非退化三角形，整数坐标绝对值 $\leq 10^6$；误差要求 | 答案-真值 |/max(1,| 真值 |)≤1e-6。此驱动使用一般多边形模板；凹多边形能力另由本地验证覆盖。当前实现已通过 P13338，记录 302630852。

所需模板：IntegerPlane（第 19 页，1.11 节）、polygon_union_area（第 32 页，1.18 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P13338

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<vector<IntegerPlane::Point>> a(n, vector<IntegerPlane::Point>(3));
    for (auto &p : a)
        for (auto &v : p) cin >> v.x >> v.y;
    cout << fixed << setprecision(12) << polygon_union_area(a) << '\n';
}
```

相关应用：圆面积并的圆弧覆盖与格林公式；支持包含、同圆、相切及孔洞，明确浮点局限。

1.19 圆面积并：未覆盖圆弧积分

问题描述：求若干圆（闭圆盘）并集的面积。

核心思路：用格林公式：并集面积等于沿并集边界的积分 $\frac{1}{2}\oint(x\,dy - y\,dx)$，而边界由每个圆上不被其他圆覆盖的圆弧组成。对每个圆，若被另一个圆包含（相同的圆只保留编号最小者）就跳过；否则求出每个相交圆在它上面覆盖的角区间，跨越 0 或 2π 时拆成两段，排序合并后对未覆盖的弧段累加闭式积分。

思路中的情况	代码中的处理	行
删点圆、分块	零半径圆无贡献；相交的圆用并查集合并，每块以代表圆心为原点，减小积分中的平移误差	29
被包含	另一圆更大且包含它，或与它完全相同且编号更小，则该圆无边界贡献	39
覆盖角区间	由三边长求交点对应的半角 α（`atan2l` 配合海伦式），区间为 $[\theta-\alpha, \theta+\alpha]$	50
弧段积分	半角 h、中点角 m 的弧贡献 $r^2h + r\sin h\,(o_x\cos m + o_y\sin m)$	74
合并区间	排序后维护已覆盖右端 `right`，空隙即未覆盖弧	84

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_union_area(a) 求闭圆盘的面积并，输入 vector<RealPlane::Circle>，圆心及半径用 long double，半径 ≥0；空集/点圆贡献 0。同圆只保留最早编号的边界，内含圆无贡献；不要求连通或无孔洞。相交连通块各自平移后积分，原输入不改变。

每个圆将其他圆覆盖的角区间切到 [0,2π]，严格排序合并，只积分未覆盖的圆弧。O(n^2 log n) 时间、O(n) 额外空间；并查集递归最深 O(n)。所有输入和中间运算须有限可表示。使用原始浮点比较，不用 eps 排序或随机扰动；近退化分类、极端尺度和最终误差仍受浮点限制。整数题域与本地对拍不构成任意实数的精确保证。

```
RealPlane::R circle_union_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = a.size();
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x) return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    R answer = 0;
    R error = 0;
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, R>> intervals;
        bool covered = false;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j) continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                covered = true;
                break;
            }
            if (d >= r + s || d <= abs(r - s)) continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0) middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                intervals.push_back({0, u});
                intervals.push_back({l + tau, tau});
            }
            else if (u > tau)
            {
                intervals.push_back({l, tau});
                intervals.push_back({0, u - tau});
            }
            else intervals.push_back({l, u});
        }
        if (covered) continue;
```

未覆盖圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error;
            R next = answer + y;
            error = (next - answer) - y;
            answer = next;
        };
        sort(intervals.begin(), intervals.end());
        R right = 0;
        for (auto [l, r] : intervals)
        {
            if (l > right) add(right, l);
            right = max(right, r);
        }
        if (right < tau) add(right, tau);
    }
    return answer;
}
```

为什么只算未覆盖弧 删去重合边界后，圆盘并边界恰是没有落在其他圆内部的圆弧。各圆逆时针方向的内部位于左侧，所以外边界和孔洞边界自动获得正确符号；不能对每条弧单独取绝对值。相切点面积为零，内含圆无需贡献。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

数值与验证 角度排序使用严格原值比较，同圆按编号去重，不以 eps 合并不同圆。半角通过缩放后的 Heron 乘积与 atan2 求出，避免 acos 靠近正负 1 时的敏感性；仍不是精确几何。独立 70 位高精度验证沿竖直截面、在全部交点横坐标处分段解析积分，另有孔洞、相切、远平移和千圆压力例。线上 AC 状态见验证记录，不能由本地通过推断。

模板题使用：SP8073 CIRU 圆面积并

题意：n≤1000，整数圆心坐标绝对值及半径 ≤1000，允许零半径。输出圆盘面积并，保留三位小数；SPOJ 转交记录 302630657 仍 Waiting。当前核心另已通过用户模板题 U106005，输出精度适配见下一用法。

所需模板：RealPlane（第 1 页，1.1 节）、circle_union_area（第 39 页，1.19 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8073

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles) cin >> c.o.x >> c.o.y >> c.r;
    cout << fixed << setprecision(3) << circle_union_area(circles) << '\n';
}
```

相关应用：分层圆弧积分求每个精确覆盖重数的面积，重复圆按编号分层；有独立高精度截面积分验证。

1.20 圆覆盖重数面积：恰好覆盖 k 次

问题描述：给定若干圆，对每个 k 求平面上恰好被 k 个圆覆盖的区域面积。

核心思路：与圆面积并一样用格林公式，但按覆盖层数分别积分。设 S_k 为至少被 k 个圆覆盖的面积。圆上一段弧若恰被 c 个圆覆盖（含自身），它就是“至少 c 层”区域的边界，于是把这段弧的积分计入 `answer[c]`。对每个圆，包含它的圆让基础层数 `count` 加一，相交的圆在角度上产生 $+1/-1$ 事件，排序扫描即得每段弧的层数。最后作差 $S_k - S_{k+1}$ 得到恰好 k 层的面积。

思路中的情况	代码中的处理	行
基础层数	自身计 1；被更大的圆包含、或与编号更小的同圆重合时再加 1，同圆因此落在不同层	45
相交圆	覆盖角区间 $[\theta-\alpha,\theta+\alpha]$ 转成 $+1$、-1 事件，跨越 0 时拆为两段；预置 0、2π 两个哨兵	33
扫描事件	相邻事件之间的弧段按当前层数积分，然后更新层数	99
按层累加	弧段积分计入 `answer[count]`，每层独立做 Kahan 补偿	88
作差	由至少 k 层面积得到恰好 k 层面积	104

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：circle_coverage_area(a) 返回长 n+1 的数组，ans[k] 为恰被 k 个闭圆盘覆盖的面积（1≤k≤n）；ans[0]=0 仅为占位，不表示无界平面面积。输入 vector<RealPlane::Circle>，圆心和半径 long double，半径 ≥0，n≤INT_MAX-2；按值传参不改原数组。支持重复圆、零半径、包含、相切和孔洞。

圆弧按覆盖层数积分得到至少 k 次面积，再相邻作差；同圆按编号依次归入不同层，不能去重。O(n^2 log n) 时间、O(n) 额外空间，并查集递归最深 O(n)。输入和中间运算须有限可表示；严格原值排序、分块平移及补偿求和不能消除近退化与小面积差分的浮点误差。不对核心结果强制取绝对值或截零。

```
vector<RealPlane::R> circle_coverage_area(vector<RealPlane::Circle> a)
{
    using R = RealPlane::R;
    R pi = acosl(-1.L);
    R tau = 2 * pi;
    for (auto c : a)
        assert(isfinite(c.o.x) && isfinite(c.o.y) && isfinite(c.r) && c.r >= 0);
    assert(a.size() <= INT_MAX - 2);
    int size = int(a.size());
    a.erase(remove_if(a.begin(), a.end(), [](auto c)
    {
        return c.r == 0;
    }), a.end());
    int n = int(a.size());
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    auto find = [&](auto &&self, int x) -> int
    {
        if (parent[x] == x)
            return x;
        return parent[x] = self(self, parent[x]);
    };
    for (int i = 0; i < n; i++)
        for (int j = 0; j < i; j++)
            if (RealPlane::norm(a[i].o - a[j].o) < a[i].r + a[j].r)
                parent[find(find, i)] = find(find, j);
    vector<R> answer(size + 2);
    vector<R> error(size + 2);
    for (int i = 0; i < n; i++)
    {
        auto c = a[i];
        auto o = c.o - a[find(find, i)].o;
        vector<pair<R, int>> events = {{0, 0}, {tau, 0}};
        int count = 1;
```

两圆关系及覆盖角区间（接上页函数）：

```
        for (int j = 0; j < n; j++)
        {
            if (i == j)
                continue;
            auto v = a[j].o - c.o;
            R d = RealPlane::norm(v);
            R r = c.r;
            R s = a[j].r;
            if (s >= r && d <= s - r && (s > r || j < i))
            {
                count++;
                continue;
            }
            if (d >= r + s || d <= abs(r - s))
                continue;
            R scale = max({d, r, s});
            d /= scale;
            r /= scale;
            s /= scale;
            R h = (r + s - d) * (d + r - s) * (d - r + s) * (d + r + s);
            R angle = atan2l(sqrtl(max(R(0), h)), d * d + (r - s) * (r + s));
            R middle = atan2l(v.y, v.x);
            if (middle < 0)
                middle += tau;
            R l = middle - angle;
            R u = middle + angle;
            if (l < 0)
            {
                events.push_back({0, 1});
                events.push_back({u, -1});
                events.push_back({l + tau, 1});
                events.push_back({tau, -1});
            }
            else if (u > tau)
            {
                events.push_back({l, 1});
                events.push_back({tau, -1});
                events.push_back({0, 1});
                events.push_back({u - tau, -1});
            }
            else
            {
                events.push_back({l, 1});
                events.push_back({u, -1});
            }
        }
```

分层圆弧积分与补偿求和（接上页函数）：

```
        auto add = [&](R l, R r)
        {
            R h = (r - l) / 2;
            R middle = (r + l) / 2;
            R value = c.r * c.r * h;
            if (r - l != tau)
                value += c.r * sinl(h) * (o.x * cosl(middle) + o.y * sinl(middle));
            R y = value - error[count];
            R next = answer[count] + y;
            error[count] = (next - answer[count]) - y;
            answer[count] = next;
        };
        sort(events.begin(), events.end());
        R left = 0;
        for (auto [right, delta] : events)
        {
            if (right > left)
                add(left, right);
            count += delta;
            left = right;
        }
    }
    for (int k = 1; k <= size; k++)
        answer[k] -= answer[k + 1];
    answer.resize(size + 1);
    return answer;
}
```

覆盖分层 当前圆弧左侧包含自身，另有 count-1 个圆覆盖；它是至少 count 次覆盖区域的正向边界。分别积分到对应层后，按 k 递增作 answer[k]-answer[k+1] 得到恰好 k 次。重合的 m 个圆按编号让第 t 个额外计入前 t-1 个同圆，从而每个阈值恰贡献一次边界；不能去重。ans[0] 只是占位。

数值与验证 各连通块平移、严格角排序、补偿累加；仍有浮点分类和相邻层相减的消去误差，小面积可能有微小负误差。466 组 70 位竖条截面积分及解析压力例，在 O2、NDEBUG、抄写版与 ASan/UBSan 下通过；不据此承诺任意实数精确。

积分公式 对相对连通块原点的圆心 (x, y)，半径 r，弧角从 l 到 u，令 $h = (u - l)/2$、$t = (u + l)/2$。格林公式给出贡献 $r^2 h + r \sin h(x \cos t + y \sin t)$。整圆直接用 πr^2，避免 $\sin \pi$ 舍入误差；补偿求和减少大量正负弧贡献相加的损失。连通块分别平移不改变闭合边界积分。

模板题使用：SP8119 CIRUT 圆覆盖重数面积

题意：n≤1000，互异圆，整数圆心坐标绝对值 ≤1000、0<r≤1000；输出恰被 k 个圆覆盖的面积，保留三位小数。核心额外支持同圆及零半径，另由本地独立验证覆盖。末端 max 仅避免舍入负零，不改变核心结果。已提交记录 302660709，仍 Waiting，不记 AC。

所需模板：RealPlane（第 1 页，1.1 节）、circle_coverage_area（第 44 页，1.20 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/SP8119

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<RealPlane::Circle> circles(n);
    for (auto &c : circles)
        cin >> c.o.x >> c.o.y >> c.r;
    auto area = circle_coverage_area(circles);
    cout << fixed << setprecision(3);
    for (int k = 1; k <= n; k++)
        cout << '[' << k << "] = " << max(0.L, area[k]) << '\n';
}
```

第二章　三维计算几何基础

相关应用：不扰动增量凸包，精确方向符号、低维退化输出、面积及六倍体积。

2.1　三维凸包：精确方向与退化处理

问题描述：求三维点集的凸包：给出外向三角面片，可计算表面积和体积；点集退化为点、线段或平面时也给出相应的边界。坐标可为整数或浮点数，方向判定精确。

核心思路：点按字典序排序去重后，依次找与 p_0, p_1 不共线的 p_c、与三者不共面的 p_d，由此确定维数。共面时丢掉法向第一个非零分量对应的坐标，投影成二维点，用单调链求二维凸包作 `boundary`，再扇形三角化。三维时以这四点建初始四面体（各面朝外），再逐点增量：删去新点能看到（方向为正）的面，被删面中反向边未被删的有向边构成地平线，与新点连成新面。方向符号整数用 `__int128` 精确算；浮点先用误差界过滤，不确定时改用浮点展开式精确求符号。

思路中的情况	代码中的处理	行
排序去重	按坐标字典序排序，相同点只保留编号最小者，`id` 记原编号	154
找不共线的第三点	`normal_sign` 三个分量全为 0 表示共线；找不到则维数为 1	167
找不共面的第四点	找不到则所有点共面，进入平面情形	171
平面情形	丢掉法向第一个非零分量对应的坐标做投影，两遍单调链得逆时针边界，再扇形三角化成 `faces`	176
初始四面体	若第四点在面的正侧就交换两顶点，使各面法向朝外	221
增量加点	可见面被删除，其有向边若反向边也被删则抵消，剩下的就是地平线	240
浮点精确符号	行列式值超过误差界直接取符号，否则用 `grow/product` 的展开式精确计算	114

接口契约：Hull3D<T>(points)，T 只取 long long 或 long double（默认）。输入编号从 0 起，不修改原数组。p 为去重后字典序点，id 映射首次原编号；faces 是 p 下标组成的外向三角面。dimension 为空/点/线/平面/三维时分别为-1/0/1/2/3；低维 boundary 给出点、线端点或平面凸多边形边界。area() 在平面时只算单面，三维时算表面积；volume6() 返回六倍体积，低维为 0。

整数坐标绝对值 $\le 10^9$，方向与六倍体积用 128 位精确运算。实数用过滤后浮点展开式确定符号，不用 eps、不扰动坐标；按已读入的二进制数求凸包，面积和实数体积仍为近似。需最近舍入、禁用 fast-math，运算及残差不得上下溢；足够安全的坐标域和具体约定见后页。

时间 $O(n^2 \log n)$、额外空间 $O(n)$。外露字段用于读取结果，不应构造后自行改写。点数与面下标须能用 int 表示。

字段与面积运算：

```
template <class T = long double>
struct Hull3D
{
    static_assert(is_same_v<T, long long> || is_same_v<T, long double>);
    using W = conditional_t<is_same_v<T, long long>, __int128, long double>;
    struct Point
    {
        T x, y, z;
    };
    vector<Point> p;
    vector<int> id, boundary;
    vector<array<int, 3>> faces;
    int dimension = -1;

    static array<W, 3> normal(Point a, Point b, Point c)
    {
        W x = W(b.x) - a.x;
        W y = W(b.y) - a.y;
        W z = W(b.z) - a.z;
        W u = W(c.x) - a.x;
        W v = W(c.y) - a.y;
        W w = W(c.z) - a.z;
        return {y * w - z * v, z * u - x * w, x * v - y * u};
    }
    static W orient(Point a, Point b, Point c, Point d)
    {
        auto v = normal(a, b, c);
        return v[0] * (W(d.x) - a.x) + v[1] * (W(d.y) - a.y) +
               v[2] * (W(d.z) - a.z);
    }
```

浮点展开式运算（接同一结构体）：

```
// Exact signs for represented floating inputs; nearest rounding, no fast-math.
// Products and residuals must neither overflow nor underflow.
using Expansion = vector<long double>;
static void grow(Expansion &a, long double q)
{
    Expansion out;
    for (long double x : a)
    {
        long double s = q + x;
        long double v = s - q;
        long double error = (q - (s - v)) + (x - v);
        if (error != 0) out.push_back(error);
        q = s;
    }
    if (q != 0) out.push_back(q);
    a.swap(out);
}
static Expansion difference(long double a, long double b)
{
    Expansion result{a};
    grow(result, -b);
    return result;
}
static Expansion product(const Expansion &a, const Expansion &b)
{
    Expansion result;
    for (long double x : a)
        for (long double y : b)
        {
            long double value = x * y;
            grow(result, fmal(x, y, -value));
            grow(result, value);
        }
    return result;
}
static int expansion_sign(const Expansion &a)
{
    if (a.empty()) return 0;
    return a.back() > 0 ? 1 : -1;
}
```

二维方向与共线检查（接同一结构体）：

```
    static int turn_sign(T x, T y, T u, T v, T s, T t)
    {
        W a = W(u) - x;
        W b = W(v) - y;
        W c = W(s) - x;
        W d = W(t) - y;
        W value = a * d - b * c;
        if constexpr (is_same_v<T, long long>)
            return (value > 0) - (value < 0);
        else
        {
            long double bound = 32 * numeric_limits<long double>::epsilon() *
                                (abs(a * d) + abs(b * c));
            if (abs(value) > bound) return value > 0 ? 1 : -1;
            auto result = product(difference(u, x), difference(t, y));
            auto subtract = product(difference(v, y), difference(s, x));
            for (long double z : subtract) grow(result, -z);
            return expansion_sign(result);
        }
    }
    static array<int, 3> normal_sign(Point a, Point b, Point c)
    {
        return {turn_sign(a.y, a.z, b.y, b.z, c.y, c.z),
                turn_sign(a.z, a.x, b.z, b.x, c.z, c.x),
                turn_sign(a.x, a.y, b.x, b.y, c.x, c.y)};
    }
```

三维方向的过滤与精确回退（接同一结构体）：

```
static int orient_sign(Point a, Point b, Point c, Point d)
{
    W value = orient(a, b, c, d);
    if constexpr (is_same_v<T, long long>)
        return (value > 0) - (value < 0);
    else
    {
        array<long double, 3> x{b.x - a.x, b.y - a.y, b.z - a.z};
        array<long double, 3> y{c.x - a.x, c.y - a.y, c.z - a.z};
        array<long double, 3> z{d.x - a.x, d.y - a.y, d.z - a.z};
        long double permanent = 0;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            permanent += (abs(x[j] * y[k]) + abs(x[k] * y[j])) * abs(z[i]);
        }
        long double bound = 128 * numeric_limits<long double>::epsilon() * permanent;
        if (abs(value) > bound) return value > 0 ? 1 : -1;
        array<Expansion, 3> ex{difference(b.x, a.x), difference(b.y, a.y), difference(b.z, a.z)};
        array<Expansion, 3> ey{difference(c.x, a.x), difference(c.y, a.y), difference(c.z, a.z)};
        array<Expansion, 3> ez{difference(d.x, a.x), difference(d.y, a.y), difference(d.z, a.z)};
        Expansion result;
        for (int i = 0; i < 3; i++)
        {
            int j = (i + 1) % 3;
            int k = (i + 2) % 3;
            auto positive = product(product(ex[j], ey[k]), ez[i]);
            auto negative = product(product(ex[k], ey[j]), ez[i]);
            for (long double v : positive) grow(result, v);
            for (long double v : negative) grow(result, -v);
        }
        return expansion_sign(result);
    }
}
```

构造、排序与去重（接同一结构体）：

```
    explicit Hull3D(const vector<Point> &input)
    {
        vector<int> order(input.size());
        iota(order.begin(), order.end(), 0);
        for (auto v : input)
        {
            if constexpr (is_same_v<T, long long>)
                assert(-1000000000 <= v.x && v.x <= 1000000000 &&
                       -1000000000 <= v.y && v.y <= 1000000000 &&
                       -1000000000 <= v.z && v.z <= 1000000000);
            else assert(isfinite(v.x) && isfinite(v.y) && isfinite(v.z));
        }
        auto key = [](Point v)
        {
            return make_tuple(v.x, v.y, v.z);
        };
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return make_tuple(input[a].x, input[a].y, input[a].z, a) <
                   make_tuple(input[b].x, input[b].y, input[b].z, b);
        });
        for (int i : order)
            if (p.empty() || key(input[i]) != key(p.back()))
            {
                p.push_back(input[i]);
                id.push_back(i);
            }
```

低维识别与平面投影（接同一构造函数）：

```
        int n = p.size();
        if (n == 0) return;
        dimension = 0;
        boundary = {0};
        if (n == 1) return;
        dimension = 1;
        boundary = {0, n - 1};
        int c = 2;
        while (c < n && normal_sign(p[0], p[1], p[c]) == array<int, 3>{}) c++;
        if (c == n) return;
        auto axis = normal_sign(p[0], p[1], p[c]);
        int d = 2;
        while (d < n && orient_sign(p[0], p[1], p[c], p[d]) == 0) d++;
        if (d == n)
        {
            dimension = 2;
            int drop = 0;
            while (axis[drop] == 0) drop++;
            auto project = [&](Point v)
            {
                if (drop == 0) return pair<T, T>{v.y, v.z};
                if (drop == 1) return pair<T, T>{v.x, v.z};
                return pair<T, T>{v.x, v.y};
            };
            auto turn = [&](int a, int b, int c)
            {
                auto [x, y] = project(p[a]);
                auto [u, v] = project(p[b]);
                auto [s, t] = project(p[c]);
                return turn_sign(x, y, u, v, s, t);
            };
```

平面单调链与三角化（接同一构造函数）：

```
        order.resize(n);
        iota(order.begin(), order.end(), 0);
        sort(order.begin(), order.end(), [&](int a, int b)
        {
            return project(p[a]) < project(p[b]);
        });
        boundary.clear();
        for (int pass = 0; pass < 2; pass++)
        {
            int base = boundary.size();
            for (int v : order)
            {
                while ((int)boundary.size() >= base + 2 &&
                       turn(boundary[boundary.size() - 2], boundary.back(), v) <= 0)
                    boundary.pop_back();
                boundary.push_back(v);
            }
            boundary.pop_back();
            reverse(order.begin(), order.end());
        }
        for (int i = 1; i + 1 < (int)boundary.size(); i++)
            faces.push_back({boundary[0], boundary[i], boundary[i + 1]});
        return;
    }
```

初始四面体与增量可见面（接同一构造函数）：

```
        dimension = 3;
        boundary.clear();
        array<int, 4> seed{0, 1, c, d};
        for (int i = 0; i < 4; i++)
        {
            array<int, 3> f{seed[(i + 1) % 4], seed[(i + 2) % 4], seed[(i + 3) % 4]};
            if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[seed[i]]) > 0)
                swap(f[1], f[2]);
            faces.push_back(f);
        }
        for (int v = 0; v < n; v++)
        {
            if (find(seed.begin(), seed.end(), v) != seed.end()) continue;
            vector<array<int, 3>> keep;
            set<pair<int, int>> horizon;
            for (auto f : faces)
            {
                if (orient_sign(p[f[0]], p[f[1]], p[f[2]], p[v]) <= 0)
                {
                    keep.push_back(f);
                    continue;
                }
                for (int k = 0; k < 3; k++)
                {
                    int a = f[k];
                    int b = f[(k + 1) % 3];
                    if (!horizon.erase({b, a})) horizon.insert({a, b});
                }
            }
            for (auto [a, b] : horizon) keep.push_back({a, b, v});
            faces.swap(keep);
        }
    }
```

面积与六倍体积（接同一结构体）：

```
    long double area() const
    {
        long double answer = 0;
        for (auto f : faces)
        {
            auto v = normal(p[f[0]], p[f[1]], p[f[2]]);
            answer += hypotl(hypotl((long double)v[0], (long double)v[1]),
                             (long double)v[2]) / 2;
        }
        return answer;
    }
    W volume6() const
    {
        W answer = 0;
        if (dimension != 3) return answer;
        for (auto f : faces) answer += orient(p[0], p[f[0]], p[f[1]], p[f[2]]);
        return answer;
    }
};
```

不变量 三维初始四面体各面均背向内部。新增点只删除严格可见的面，删除面中的反向公共边两两消去，剩余有向边是轮廓。用轮廓边和新点组成新面即可恢复闭合外向凸包；零方向不算可见，不能扰动共面点。所有点先排序去重，平面情况投影到法向非零的轴，单独用二维凸包处理。

精确符号的范围 浮点展开式保存每次加法和乘法被舍弃的残差，末项符号即精确和的符号；乘法残差通过 fma 计算。先用保守舍入误差界过滤明显非零的方向，不能确定时回退。需二进制、最近舍入、禁用 fast-math 及运算重排，保证运算和残差不上下溢。对于有效位不超过 113 且指数范围至少覆盖普通 double 的实现，每个非零坐标绝对值在 2 的负 100 次方至 2 的 100 次方内是一个足够安全域。输入十进制读取误差不被恢复。

测量和退化约定 精确方向不等于精确面积。面积的平方根与求和、实数体积仍可能失去相对精度，极薄模型需额外检验。整数域内六倍体积精确且非负；平面核心只返回单面面积，点和线面积为 0。题目驱动选择平面双面包裹约定，题面本身没有明确低维语义；其他题应按题意适配。

独立验证 565 组输入分别验证整数和实数版，参考为精确支撑平面枚举及每个面的二维凸包；检查外向、无退化面、反向边配对、Euler 关系、面积和体积。另有 8000 组精确有理数方向参考，含共面、nextafter 近共面、不同指数和减法残差。原先放大整数点导致浮点行列式误判的反例保留。

模板题使用：P4724 三维凸包

题意：4≤n≤2000 个实数点，输出三维凸包表面积，保留三位小数。题面未给坐标界。驱动对纯平面输入采用正反双面包裹面积约定；核心 area() 仍返回平面单面面积。线上状态见验证记录。

所需模板：Hull3D（第 49 页，2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4724

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<Hull3D<>::Point> p(n);
    for (auto &v : p) cin >> v.x >> v.y >> v.z;
    Hull3D hull(p);
    long double area = hull.area();
    if (hull.dimension == 2) area *= 2;
    cout << fixed << setprecision(3) << area << '\n';
}
```

2.2 整数三维共线、共面与方向

功能： 整数坐标的三维基础判定：四点有向体积（方向，0 表示共面）、三点共线、点是否在线段上。

核心思路： 坐标差扩展到 `__int128_t` 后计算。`orient(a,b,c,d)` 为混合积 $((b-a)\times(c-a))\cdot(d-a)$，即四面体有向体积的 6 倍，正值表示 d 在 $(b-a)\times(c-a)$ 所指的一侧。`collinear` 判断 $(b-a)\times(c-a)$ 是否为零向量；`on_segment` 在共线基础上要求 $(a-p)\cdot(b-p)\le 0$。

接口契约： 坐标绝对值 $\le 10^9$；orient(a,b,c,d) 为有向六倍四面体体积，0 表示共面；collinear 判三点共线，on_segment 支持退化线段。int128 精确判定，不包含三维凸包构造。

```
struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};
```

2.3 实数三维向量与夹角

功能：浮点三维向量基础：加减、数乘数除、字典序比较，以及点积、叉积、长度、单位化和两向量夹角。

核心思路：`norm` 用三参数 `hypot` 求长度，避免先平方溢出；`unit` 除以长度得单位向量。`angle` 先把两向量单位化，再用 atan2($|a \times b|, a \cdot b$) 求夹角，比 arccos 在夹角接近 0 或 π 时更稳定。

接口契约：Point 为 long double 三坐标，支持加减、数乘/除和精确字典序；排序禁用 epsilon。dot、cross、norm 为点积、叉积、长度；unit 和 angle 要求非零向量，angle 返回 0..π 的有向向量夹角，弧度制。unit(a)*r 变换长度；点间距离为 norm(a-b)。有限输入且中间量不溢出/下溢，极端尺度须预缩放；浮点不是精确共面判定。O(1) 时间空间。

```
struct RealSpace
{
    using R = long double;

    struct Point
    {
        R x = 0, y = 0, z = 0;

        Point operator+(Point b) const
        {
            return {x + b.x, y + b.y, z + b.z};
        }

        Point operator-(Point b) const
        {
            return {x - b.x, y - b.y, z - b.z};
        }

        Point operator*(R k) const
        {
            return {x * k, y * k, z * k};
        }

        Point operator/(R k) const
        {
            return {x / k, y / k, z / k};
        }

        bool operator<(Point b) const
        {
            return tie(x, y, z) < tie(b.x, b.y, b.z);
        }
    };
```

继续三维运算（接上页同一结构体）：

```
    static R dot(Point a, Point b)
    {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }

    static Point cross(Point a, Point b)
    {
        return {a.y * b.z - a.z * b.y,
                a.z * b.x - a.x * b.z,
                a.x * b.y - a.y * b.x};
    }

    static R norm(Point a)
    {
        return hypot(a.x, a.y, a.z);
    }

    static Point unit(Point a)
    {
        R len = norm(a);
        assert(len > 0);
        return a / len;
    }

    // Directed vectors: angle in [0, pi], both must be nonzero.
    static R angle(Point a, Point b)
    {
        a = unit(a);
        b = unit(b);
        return atan2(norm(cross(a, b)), dot(a, b));
    }
};

struct IntegerGeometry3D
{
    using I = __int128_t;

    // |coordinates|<=1e9: triple products fit signed int128.
    struct Point
    {
        long long x, y, z;
    };

    struct Vector
    {
        I x, y, z;
    };

    static Vector diff(Point a, Point b)
    {
        return {I(a.x) - b.x, I(a.y) - b.y, I(a.z) - b.z};
    }

    static Vector cross(Vector a, Vector b)
```

```
    {
        return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
    }

    static I dot(Vector a, Vector b) { return a.x * b.x + a.y * b.y + a.z * b.z; }

    static I orient(Point a, Point b, Point c, Point d)
    {
        return dot(cross(diff(b, a), diff(c, a)), diff(d, a));
    }

    static bool collinear(Point a, Point b, Point c)
    {
        auto v = cross(diff(b, a), diff(c, a));
        return !v.x && !v.y && !v.z;
    }

    static bool on_segment(Point p, Point a, Point b)
    {
        return collinear(p, a, b) && dot(diff(a, p), diff(b, p)) <= 0;
    }
};

struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }

    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
```

```
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};

struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }

    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
```

```
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

2.4 三维直线投影、线段距离与绕轴旋转

功能：三维直线 $p+td$（线段取 $0 \le t \le 1$）：求点到直线或线段的投影与距离，判断点是否在线段上，以及把点绕该直线旋转。

核心思路：投影先把方向单位化为 u，取 $p+u\,((q-p)\cdot u)$；线段版把投影长度截断到 $[0,|d|]$。点到直线距离为 $|u\times(q-p)|$，到线段距离为到截断后投影点的距离。`rotate` 用 Rodrigues 公式：令 $v=q-p$，旋转后为 $v\cos\theta+(u\times v)\sin\theta+u\,(u\cdot v)(1-\cos\theta)$，再加回 p。

代码依赖：RealSpace（第 61 页，2.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Line3(p,d) 表示 p+t*d，注意第二参数是方向；through(a,b) 从两个端点构造。线段限制 0≤t≤1。projection/segment_projection 返回最近点，distance/segment_distance 返回距离，on_segment(q,eps) 使用绝对距离容差。d=0 时这些操作按点 p 处理；rotate(q,rad) 必须 d 非零，绕 p+t*d 按右手法则旋转，角度单位弧度。公开 p,d。依赖 RealSpace；O(1)。近退化浮点运算须按输入尺度分析误差。

```
struct Line3 : RealSpace
{
    Point p, d; // p + t*d; segment uses 0 <= t <= 1.

    Line3(Point p = {}, Point d = {}) : p(p), d(d)
    {
    }

    static Line3 through(Point a, Point b)
    {
        return Line3(a, b - a);
```

```
    }

    Point projection(Point q) const
    {
        if (norm(d) == 0) return p;
        Point u = unit(d);
        return p + u * dot(q - p, u);
    }

    Point segment_projection(Point q) const
    {
        R len = norm(d);
        if (len == 0) return p;
        Point u = d / len;
        R t = clamp(dot(q - p, u), R(0), len);
        return p + u * t;
    }
```

继续三维运算（接上页同一结构体）：

```
    R distance(Point q) const
    {
        if (norm(d) == 0) return norm(q - p);
        return norm(cross(unit(d), q - p));
    }

    R segment_distance(Point q) const
    {
        return norm(q - segment_projection(q));
    }

    bool on_segment(Point q, R eps = 1e-10L) const
    {
        assert(eps >= 0);
        return segment_distance(q) <= eps;
    }

    // Right-hand rotation about p + t*d; d must be nonzero.
    Point rotate(Point q, R rad) const
    {
        Point u = unit(d), v = q - p;
        R c = cos(rad), s = sin(rad);
        return p + v * c + cross(u, v) * s + u * (dot(u, v) * (1 - c));
    }
};

struct Plane3 : RealSpace
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
```

```
    }

    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

2.5 三维平面投影与分类求交

功能：三维平面（点 p 与单位法向 n）：可由三点或方程 $n \cdot x = h$ 构造，求有符号距离与投影，以及平面与直线、平面与平面的求交和分类。

核心思路：有符号距离为 $n \cdot (q - p)$，投影为 q 减去该距离乘 n。直线求交：方向单位化为 u，$t = n \cdot u$ 是直线与平面夹角的正弦，$|t|$ 不超过 ang 时视为平行，再按起点到平面的距离区分在面上或无交；否则交点为起点减 $u\,h/t$（h 为起点的有符号距离）。两平面求交：交线方向 $d = n_1 \times n_2$，过短则平行或重合；否则在本平面内沿垂直于 d 的 $v = d \times n_1$ 走到另一平面，得到交线上一点。

代码依赖：Line3（第 65 页，2.4 节）、RealSpace（第 61 页，2.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Plane3(p,normal) 要求非零法向量并单位化；through(a,b,c) 要求三点不共线；equation(n,h) 表示 n 点乘 x=h。signed_distance 保留法向符号，projection 返回垂足。intersect(line,q,eps,ang) 返回 1 单点、0 无交、-1 整条非退化线在面上，退化线按点处理。intersect(plane,line,eps,ang) 返回 1 交线、0 平行不同、-1 重合。仅返回 1 时写输出参数；交线 d 为单位向量。eps 为距离容差，ang 为单位方向的正弦容差；默认 1e-10 和 1e-12。近乎平行按容差分类，非精确拓扑判定；近乎平行求交误差会放大。公开 p 和单位 n，勿破坏单位不变量。依赖 RealSpace/Line3；O(1)。

```
struct Plane3 : RealSpace
```

```
{
    Point p, n; // n is a unit normal; dot(n, x - p) = 0.

    Plane3(Point p, Point normal) : p(p), n(unit(normal))
    {
    }

    static Plane3 through(Point a, Point b, Point c)
    {
        return Plane3(a, cross(b - a, c - a));
    }

    // dot(normal, x) = h; normal must be nonzero.
    static Plane3 equation(Point normal, R h)
    {
        Point u = unit(normal);
        return Plane3(u * (h / norm(normal)), u);
    }

    R signed_distance(Point q) const
    {
        return dot(n, q - p);
    }

    Point projection(Point q) const
    {
        return q - n * signed_distance(q);
    }
```

继续三维运算（接上页同一结构体）：

```
    // 1: one point; 0: none; -1: the nondegenerate line lies in plane.
    int intersect(Line3 l, Point &q, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        R h = signed_distance(l.p);
        if (norm(l.d) == 0)
        {
            if (abs(h) > eps) return 0;
            q = l.p;
            return 1;
        }
        Point u = unit(l.d);
        R t = dot(n, u);
        if (abs(t) <= ang) return abs(h) <= eps ? -1 : 0;
        q = l.p - u * (h / t);
        return 1;
    }

    // 1: one line; 0: distinct parallel planes; -1: coincident planes.
    int intersect(Plane3 f, Line3 &l, R eps = 1e-10L, R ang = 1e-12L) const
    {
        assert(eps >= 0 && 0 <= ang && ang < 1);
        Point d = cross(n, f.n);
        R len = norm(d);
        if (len <= ang) return abs(signed_distance(f.p)) <= eps ? -1 : 0;
        d = d / len;
        Point v = cross(d, n);
        Point q = p + v * (dot(f.n, f.p - p) / dot(f.n, v));
        l = Line3(q, d);
        return 1;
    }
};
```

第三章　距离

3.1　点到闭线段的距离

功能：求点 p 到闭线段 ab 的最短距离。

核心思路：把 p 投影到直线 ab 上，得参数 $t = \frac{(p-a)\cdot v}{v\cdot v}$（$v = b - a$），再截断到 $[0, 1]$：投影落在线段外时最近点就是较近的端点。返回 p 到 $a + tv$ 的距离；$a = b$ 时退化为到该点的距离。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：segment_distance_real(p,a,b) 返回点 p 到闭线段 ab 的 long double 距离。a=b 时返回到端点的距离；否则把投影参数截断至 [0,1]。要求有限且尺度适当的输入；O(1) 时间和空间。

```
RealPlane::R
segment_distance_real(RealPlane::Point p, RealPlane::Point a, RealPlane::Point b)
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    Point v = b - a;
    R d = G::dot(v, v);
    if (d == 0) return G::norm(p - a);
    R t = clamp(G::dot(p - a, v) / d, R(0), R(1));
    return G::norm(p - (a + v * t));
}
```

模板题使用：AOJ CGL_2_D

题意：求两条闭线段间距离，q 不超过 1000，整数坐标绝对值不超过 10000。先用 IntegerPlane 精确判相交（含接触和共线重叠），相交输出 0；否则取四个端点到另一线段距离的最小值。单点距离接口参数为 (p,a,b)。题目线段非退化，点线段属于额外接口边界。

所需模板：IntegerPlane（第 19 页，1.11 节）、RealPlane（第 1 页，1.1 节）、segment_distance_real（第 71 页，3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://onlinejudge.u-aizu.ac.jp/problems/CGL_2_D

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q;
    cin >> q;
    cout << fixed << setprecision(12);
    while (q--)
    {
        vector<IntegerPlane::Point> p(4);
        vector<RealPlane::Point> a(4);
        for (int i = 0; i < 4; i++)
        {
            cin >> p[i].x >> p[i].y;
            a[i] = {(long double)p[i].x, (long double)p[i].y};
        }
        long double ans = 0;
        if (!IntegerPlane::intersect(p[0], p[1], p[2], p[3]))
            ans = min({segment_distance_real(a[0], a[2], a[3]),
                       segment_distance_real(a[1], a[2], a[3]),
                       segment_distance_real(a[2], a[0], a[1]),
                       segment_distance_real(a[3], a[0], a[1])});
        cout << ans << '\n';
    }
}
```

第四章　Pick 定理

4.1　Pick 定理：格点、剖分与孔洞

简单、非退化、整数顶点、无孔多边形，A 为面积，I 为严格内部格点数，B 为边界格点数：

$$A = I + \frac{B}{2} - 1, \qquad I = \frac{|S| - B + 2}{2}, \qquad S = \sum_i (x_i y_{i+1} - y_i x_{i+1}),$$

$$B = \sum_i \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|).$$

对总和 S 取绝对值，不对单项取。有向面积见第 22 页。有 h 个互不接触、不嵌套的简单孔洞时 $A = I + \frac{B}{2} - 1 + h$，$B$ 含所有内外环，I 不含孔洞内部。非整数顶点、自交、边界相接或退化均不适用；面积非负不能代替简单性检查。坐标 $|x|, |y| \le 10^{12}$、$n \le 10^5$ 时叉积、S 与分子用 128 位。

第五章　凸包

5.1　凸多边形外点的两条切线

问题描述：给定严格凸、逆时针的整数多边形和一点 q，若 q 在多边形外，求从 q 向多边形所作两条切线的切点下标（右切点、左切点）；q 在多边形内或边界上时报告无切线。

核心思路：称边 i（$p_i \to p_{i+1}$）可见，若 q 严格在它右侧（`visible`）。从 q 看，可见边在环上连成一段，两个切点正是可见与不可见交替处的顶点。代码先借助 p_0 的两条邻边和以 p_0 为中心的扇形二分（`sector`），各找出一条可见边 v 与一条不可见边 h；于是环上从 h 到 v、从 v 到 h 两段的可见性都只切换一次，各用一次二分（`boundary`）找到切换处的顶点，即右、左切点。

思路中的情况	代码中的处理	行
边 0 可见	取 $v=0$；边 $n-1$ 不可见则 $h=n-1$，否则 q 在 p_0 内角的对顶角内，射线 $q \to p_0$ 穿出多边形的那条边不可见，用 `sector(-1)` 二分找到它	27
只有边 $n-1$ 可见	$v=n-1$，$h=0$	32
两条邻边都不可见	q 在 p_0 的内角中，`sector(1)` 找到包含 q 方向的扇形，其外侧边若不可见则 q 在多边形内或边界上，返回空	39
二分切换点	从 `start` 沿环二分第一条可见性与 `start` 不同的边，返回该边起点	46
输出	$h \to v$ 段的切换点为右切点，$v \to h$ 段的为左切点	57

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：p 须为严格凸、逆时针的整数多边形，无重复末点或共线边界顶点；3≤n≤INT_MAX/2，起始下标任意。所有顶点及 q 坐标绝对值 ≤10^12。返回 optional<pair<int,int» 的原 0-based 下标 (right,left)：对任意顶点 v，cross(q,p[right],v)≤0，cross(q,p[left],v)≥0。若切线贴整条边，选离 q 最近的端点。q 在内部或边界时返回 nullopt。O(log n) 时间、O(1) 额外空间，无预处理且输入不变；先扇形定位异态边，再二分可见边链两端。凸性、方向及数值范围均为前置条件，assert 不验证整个多边形；n<3 不在接口范围。散点先构造 integer_hull，若需原编号须另留映射。

```
optional<pair<int, int>> convex_tangents_i64(const vector<IntegerPlane::Point> &p,
                                             IntegerPlane::Point q)
{
    using G = IntegerPlane;
    assert(p.size() >= 3 && p.size() <= INT_MAX / 2);
    int n = p.size();
    auto visible = [&](int i)
    {
        return G::cross(p[i], p[(i + 1) % n], q) < 0;
    };
    auto sector = [&](int dir)
    {
        int l = 1, r = n - 1;
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (dir * G::cross(p[0], p[m], q) >= 0)
                l = m;
            else
                r = m;
        }
        return l;
    };
```

选择异态种子边并二分可见边链（接上页同一函数）：

```
    int a = G::sign(G::cross(p[0], p[1], q));
    int b = G::sign(G::cross(p[0], p[n - 1], q));
    int v, h;
    if (a < 0)
    {
        v = 0;
        h = b <= 0 ? n - 1 : sector(-1);
    }
    else if (b > 0)
    {
        v = n - 1;
        h = 0;
    }
    else
    {
        v = sector(1);
        h = 0;
        if (!visible(v)) return nullopt;
    }
    auto boundary = [&](int start, int end)
    {
        int l = 0, r = (end - start + n) % n;
        bool first = visible(start);
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            if (visible((start + m) % n) == first)
                l = m;
            else
                r = m;
        }
        return (start + r) % n;
    };
    return pair{boundary(h, v), boundary(v, h)};
}
```

5.2 点在整数凸包内（二分）

问题描述：判断点 q 在严格逆时针凸多边形的外部、边界上还是内部，每次询问对数时间。

核心思路：以 p_0 为中心把凸多边形剖成扇形三角形。先看 q 是否在射线 p_0p_1 与 p_0p_{n-1} 夹成的角内：在角外即为外部；恰在其中一条边所在直线上时，按是否落在对应边上判边界或外部。否则二分出 q 所在扇区，即满足 $(p_l - p_0) \times (q - p_0) \geq 0$ 的最大 l，最后由 q 相对边 p_lp_{l+1} 的叉积符号给出外部、边界或内部。

代码依赖：IntegerPlane（第 19 页，1.11 节）、polygon_contains（第 23 页，1.13 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_contains_i64(p,q) 返回 0 外部、1 边界、2 内部。要求严格逆时针凸包，无共线边内点、不重复首点，例如 integer_hull 的输出；允许循环旋转起点。空集、单点、线段转用 polygon_contains。坐标范围同 IntegerPlane，O(log n) 时间、O(1) 空间。

```
int convex_contains_i64(const vector<IntegerPlane::Point> &p, IntegerPlane::Point q)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (n < 3) return polygon_contains(p, q);
    I a = G::cross(p[0], p[1], q), b = G::cross(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return 0;
    if (!a) return G::on_segment(q, p[0], p[1]) ? 1 : 0;
    if (!b) return G::on_segment(q, p[0], p[n - 1]) ? 1 : 0;
    int l = 1, r = n - 1;
    while (l + 1 < r)
    {
        int m = (l + r) / 2;
        if (G::cross(p[0], p[m], q) >= 0)
            l = m;
        else
            r = m;
    }
    I c = G::cross(p[l], p[r], q);
    return c < 0 ? 0 : c == 0 ? 1 : 2;
}
```

5.3 整数凸包：单调链

问题描述：求平面整数点集的凸包，按逆时针输出顶点，去掉重复点和边上的非顶点。

核心思路：Andrew 单调链。点按 (x, y) 排序去重后，从左到右扫描构造下凸壳：栈顶两点与新点不构成严格左转（叉积 ≤ 0）就弹栈；再从右到左扫描构造上凸壳，弹栈时不越过下凸壳的部分（`low`）。两条链首尾相接，最后弹出重复的起点。因为叉积为 0 也弹栈，共线点都被删去，全共线时只剩两个端点。

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：integer_hull(points) 按值接收 IntegerPlane::Point 序列，输出严格逆时针凸包，不重复首点、不保留共线边内点。自动排序去重；全共线时保留两个端点，空集及单点直接返回。坐标范围同 IntegerPlane。O(n log n) 时间、O(n) 空间；供凸包包含、直径与 Minkowski 和调用。

```
vector<IntegerPlane::Point> integer_hull(vector<IntegerPlane::Point> p)
{
    using G = IntegerPlane;
    using Point = G::Point;
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    int n = (int)p.size();
    if (n < 3) return p;
    vector<Point> h;
    for (Point v : p)
    {
        while (h.size() > 1 && G::cross(h[h.size() - 2], h.back(), v) <= 0)
            h.pop_back();
        h.push_back(v);
    }
    int low = (int)h.size();
    for (int i = n - 2; i >= 0; i--)
    {
        while ((int)h.size() > low && G::cross(h[h.size() - 2], h.back(), p[i]) <= 0)
            h.pop_back();
        h.push_back(p[i]);
    }
    h.pop_back();
    return h; // CCW, no repeated endpoint, no collinear interior points.
}
```

模板题使用：Library Checker static_convex_hull

题意：多组求整数点集的凸包，去重并去掉边上非顶点。输出顶点数及逆时针顶点，不重复首点；空集输出 0，共线时仅保留两端。传 move 交出点集，避免按值传参时额外复制。

所需模板：IntegerPlane（第 19 页，1.11 节）、integer_hull（第 78 页，5.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/static_convex_hull

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(move(points));
        cout << hull.size() << '\n';
        for (auto p : hull) cout << p.x << ' ' << p.y << '\n';
    }
}
```

相关应用：固定导航没有独立 Minkowski 和条目，按凸包的相关应用归档。

5.4 凸多边形 Minkowski 和

问题描述：给定两个凸多边形 A, B，求 Minkowski 和 $A + B = \{a + b \mid a \in A, b \in B\}$ 的顶点（逆时针）。

核心思路：$A + B$ 的边恰是 A、B 全部边按极角归并的结果。先把两个多边形各自旋转到以最低（y 最小，其次 x 最小）顶点开头，此时两者的边都从同一极角起逆时针排列；从 $a_0 + b_0$ 出发，每步输出当前和点，用叉积比较两条当前边，前进极角较小的一侧，平行时两侧同时前进。点或线段输入不走归并，直接枚举所有和点再调用 `integer_hull`。

代码依赖：IntegerPlane（第 19 页，1.11 节）、integer_hull（第 78 页，5.3 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：minkowski_sum(a,b) 接受不重复首点的严格逆时针凸包；空输入返回空集，点和线段允许作为退化凸包。

点类型 IntegerPlane::Point；输入坐标及两集合坐标和的绝对值不超过 10^{12}。结果不重复首点，非退化时沿逆时针边方向合并，平行同向边同时推进；O(n+m) 时间。

退化输入枚举与另一凸包各点的和再求凸包，O((n+m) log(n+m))。按值传入，内部旋转不修改调用方；依赖 IntegerPlane 的 integer_hull/cross。

```cpp
vector<IntegerPlane::Point> minkowski_sum(vector<IntegerPlane::Point> a,
                                          vector<IntegerPlane::Point> b)
{
    using G = IntegerPlane;
    using P = G::Point;
    using I = G::I;
    if (a.empty() || b.empty()) return {};
    auto plus = [](P u, P v)
    {
        return P{u.x + v.x, u.y + v.y};
    };
    if (a.size() < 3 || b.size() < 3)
    {
        vector<P> p;
        for (P u : a)
            for (P v : b) p.push_back(plus(u, v));
        return integer_hull(p);
    }
    auto start = [](vector<P> &p)
    {
        auto it = min_element(
            p.begin(), p.end(), [](P u, P v) { return tie(u.y, u.x) < tie(v.y, v.x); });
        rotate(p.begin(), it, p.end());
    };
    start(a);
    start(b);
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> result;
```

合并凸包边（接上页同一函数）：

```
    while (i < n || j < m)
    {
        result.push_back(plus(a[i % n], b[j % m]));
        if (i == n)
        {
            ++j;
            continue;
        }
        if (j == m)
        {
            ++i;
            continue;
        }
        P u{a[(i + 1) % n].x - a[i].x, a[(i + 1) % n].y - a[i].y};
        P v{b[(j + 1) % m].x - b[j].x, b[(j + 1) % m].y - b[j].y};
        I c = G::cross({0, 0}, u, v);
        if (c >= 0) ++i;
        if (c <= 0) ++j;
    }
    return result;
}
```

第六章　扫描线

6.1　整数轴对齐矩形面积并

问题描述：给定若干轴对齐矩形（按左、下、右、上给出整数坐标），求它们并集的面积，重叠部分只计一次。

核心思路：扫描线沿 x 方向推进。每个矩形产生进入事件 $(l, d, u, +1)$ 与离开事件 $(r, d, u, -1)$，按横坐标排序；纵坐标离散化后，线段树叶子代表相邻两坐标之间的区间。节点 `cover` 记录该区间被整段覆盖的次数（不下传），`len` 为区间内被覆盖的总长度：`cover` 非零时取整段长度，否则取两孩子之和。处理每个事件前，先把与上一事件的横坐标差乘以根的覆盖长度 `len[1]` 累加进面积。

接口契约：调用 rectangle_union_area(rect)，每项按左、下、右、上存储四个 long long。端点有序，各坐标绝对值不超过 10^{18}；零面积矩形忽略，空集返回 0。返回精确 ___int128 面积；边界接触不增加面积，输入不变，多次调用独立。扫描线处理进出事件，纵向叶子代表相邻坐标间的半开区间。cover 仅记录整段覆盖次数，不下传；非零时取整段长度，为零时合并孩子。同横坐标事件间宽度为 0。时间 O(n log(n+1))、空间 O(n)，递归深度 O(log(n+1))；矩形数至多 268435454，且内存可用。题目坐标在 0 到 10^9，故驱动可将结果转 long long；一般输入不要直接窄化。不含周长、浮点或动态更新。

```
__int128 rectangle_union_area(const vector<array<long long, 4>> &rect)
{
    using ll = long long;
    assert(rect.size() <= (INT_MAX - 8) / 8);
    vector<array<ll, 4>> events;
    vector<ll> y;
    for (auto [l, d, r, u] : rect)
    {
        for (ll x : {l, d, r, u})
            assert(-1000000000000000000LL <= x && x <= 1000000000000000000LL);
        assert(l <= r && d <= u);
        if (l == r || d == u) continue;
        events.push_back({l, d, u, 1});
        events.push_back({r, d, u, -1});
        y.push_back(d);
        y.push_back(u);
    }
    if (events.empty()) return 0;
    sort(events.begin(), events.end());
    sort(y.begin(), y.end());
    y.erase(unique(y.begin(), y.end()), y.end());
    int n = int(y.size()) - 1;
    vector<int> cover(4 * n + 4);
    vector<ll> len(4 * n + 4);
```

覆盖长度维护与横向面积累加（接上页同一函数）：

```
    auto add = [&](auto &&self, int p, int l, int r, int a, int b, int v) -> void
    {
        if (a <= l && r <= b) cover[p] += v;
        else
        {
            int m = (l + r) / 2;
            if (a < m) self(self, p * 2, l, m, a, b, v);
            if (b > m) self(self, p * 2 + 1, m, r, a, b, v);
        }
        if (cover[p]) len[p] = y[r] - y[l];
        else len[p] = r - l == 1 ? 0 : len[p * 2] + len[p * 2 + 1];
    };
    __int128 area = 0;
    ll previous = events.front()[0];
    for (auto [x, d, u, v] : events)
    {
        area += (__int128)(x - previous) * len[1];
        int l = lower_bound(y.begin(), y.end(), d) - y.begin();
        int r = lower_bound(y.begin(), y.end(), u) - y.begin();
        add(add, 1, 0, n, l, r, int(v));
        previous = x;
    }
    return area;
}
```

模板题使用：Library Checker Area of Union of Rectangles

题意：输入 N 个轴对齐矩形，每行依次为 l,d,r,u。1≤N≤500000，0≤l<r≤10^9，0≤d<u≤10^9。直接按左、下、右、上存储，计算并集面积；不是面积和。所有矩形都在边长 10^9 的正方形内，最终面积 ≤10^18，可把 __int128 结果转成 long long 输出。

所需模板：rectangle_union_area（第 82 页，6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/area_of_union_of_rectangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<array<long long, 4>> rect(n);
    for (auto &a : rect) for (auto &x : a) cin >> x;
    // The statement bounds the whole union by [0,1e9]^2.
    cout << (long long)rectangle_union_area(rect) << '\n';
}
```

第七章　旋转卡壳

7.1　整数凸包直径：旋转卡壳

问题描述：求凸多边形顶点间最远距离的平方（平面点集的直径，散点先求凸包），可同时给出一对最远点的下标。

核心思路：旋转卡壳。按逆时针枚举每条边 p_ip_{i+1}，指针 j 指向离这条边所在直线最远的顶点：只要 p_{j+1} 与该边构成的三角形面积（叉积）更大就前移 j。边逆时针转动时 j 也单调前移，总共只绕一圈。最远点对必出现在某条边的端点与其对踵点之间，故每次用 (p_i, p_j)、(p_{i+1}, p_j) 更新答案。

代码依赖：IntegerPlane（第 19 页，1.11 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：convex_diameter2(p,endpoints=nullptr) 返回最远两点距离平方，类型 ___int128_t；不足两点返回 0。输入须为严格逆时针凸包，不重复首点，例如 integer_hull 的输出。坐标范围同 IntegerPlane；旋转卡壳 O(n) 时间、O(1) 额外空间。可传 pair<int,int> 的地址，接收凸包内的 0-based 端点下标；空集为 {-1,-1}，单点为 {0,0}，至少两点时端点不同。不保证并列最优端点的顺序。映射到原始输入编号由调用者完成。

```
IntegerPlane::I convex_diameter2(const vector<IntegerPlane::Point> &p,
                                 pair<int, int> *endpoints = nullptr)
{
    using G = IntegerPlane;
    using I = G::I;
    int n = (int)p.size();
    if (endpoints) *endpoints = n ? pair{0, n - 1} : pair{-1, -1};
    if (n < 2) return 0;
    if (n == 2) return G::dist2(p[0], p[1]);
    I ans = 0;
    auto update = [&](int a, int b)
    {
        I value = G::dist2(p[a], p[b]);
        if (value > ans)
        {
            ans = value;
            if (endpoints) *endpoints = {a, b};
        }
    };
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        int k = (i + 1) % n;
        while (G::cross(p[i], p[k], p[(j + 1) % n]) > G::cross(p[i], p[k], p[j]))
            j = (j + 1) % n;
        update(i, j);
        update(k, j);
    }
    return ans;
}
```

模板题使用：Library Checker furthest_pair

题意：多组求距离最远的两个不同输入点编号。先建凸包，端点接口返回凸包下标，再按坐标映射回输入下标。题目保证至少两点；若凸包只剩一个点，所有坐标相同，输出 0、1。

所需模板：IntegerPlane（第 19 页，1.11 节）、integer_hull（第 78 页，5.3 节）、convex_diameter2（第 84 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/furthest_pair

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int tests;
    cin >> tests;
    while (tests--)
    {
        int n;
        cin >> n;
        vector<IntegerPlane::Point> points(n);
        for (auto &p : points) cin >> p.x >> p.y;
        auto hull = integer_hull(points);
        if (hull.size() == 1)
        {
            cout << "0 1\n";
            continue;
        }
        pair<int, int> ends;
        convex_diameter2(hull, &ends);
        int a = -1, b = -1;
        for (int i = 0; i < n; i++)
        {
            if (points[i] == hull[ends.first]) a = i;
            if (points[i] == hull[ends.second]) b = i;
        }
        cout << a << ' ' << b << '\n';
    }
}
```

相关应用：凸包的三组支撑点旋转卡壳，最小包围矩形面积与逆时针四角。

7.2 最小面积包围矩形：旋转卡壳与四角

问题描述：给定凸多边形（严格逆时针凸包），求面积最小的包围矩形，返回面积与四个角点。

核心思路：最优矩形必有一边与凸包某条边共线。枚举每条边 p_ip_{i+1} 作底边方向 e，维护三个随边单调前移的指针：沿 e 投影最大的 `hi`、最小的 `lo`，以及离底边最远的 `top`。设 l, r 为两端的投影、h 为高（都乘了 $|e|$），矩形面积为 $(r-l)h/|e|^2$。更新最优解时，两个底角为 $p_i + e\,l/|e|^2$、$p_i + e\,r/|e|^2$，再加上高向量 $\mathrm{perp}(e)\,h/|e|^2$ 得另两角。首条边的三个指针先线性扫描初始化。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：minimum_rectangle(p) 返回 pair(area,box)，box 是 array<Point,4>，非退化时按逆时针列出最优矩形四角；并列时不指定首点或方向。输入 RealPlane::Point 严格逆时针凸包，不重复首点、不保留共线中间点，不能直接传散点或顺时针多边形。空集返回 0 和四个原点；单点重复四次，线段返回 a,b,b,a；原输入不变。

沿每条凸包边维护前后投影极值与最大高度，恢复四条支撑线的交点。精确支撑比较下 O(n) 时间、O(1) 额外空间；n≤INT_MAX，输入及所有中间运算须有限可表示。浮点版不使用 eps 或扰动，近退化符号及角点精度仍受舍入影响；steps<n 是异常平坦比较的终止保护，不能修复舍入错误（异常时最坏 $O(n^2)$）。有界整数/定点题优先精确构造凸包，再传实数坐标。

```
pair<RealPlane::R, array<RealPlane::Point, 4>> minimum_rectangle(
    const vector<RealPlane::Point> &p)
{
    using G = RealPlane;
    using R = G::R;
    using P = G::Point;
    int n = int(p.size());
    if (n == 0)
        return {0, {}};
    if (n == 1)
        return {0, {p[0], p[0], p[0], p[0]}};
    if (n == 2)
        return {0, {p[0], p[1], p[1], p[0]}};
    int hi = 0;
    int lo = 0;
    int top = 0;
    P e = p[1] - p[0];
    for (int j = 1; j < n; j++)
    {
        if (G::dot(e, p[j] - p[hi]) > 0)
            hi = j;
        if (G::dot(e, p[j] - p[lo]) < 0)
            lo = j;
        if (G::cross(e, p[j] - p[top]) > 0)
            top = j;
    }
    R best = numeric_limits<R>::infinity();
    array<P, 4> box;
```

三组支撑点推进与四角恢复（接上页函数）：

```
    for (int i = 0; i < n; i++)
    {
        e = p[(i + 1) % n] - p[i];
        auto advance = [&](int &j, auto better)
        {
            int steps = 0;
            while (steps < n && better(p[(j + 1) % n] - p[j]))
            {
                j = (j + 1) % n;
                steps++;
            }
        };
        advance(hi, [&](P v)
        {
            return G::dot(e, v) >= 0;
        });
        advance(lo, [&](P v)
        {
            return G::dot(e, v) <= 0;
        });
        advance(top, [&](P v)
        {
            return G::cross(e, v) >= 0;
        });
        R d = G::dot(e, e);
        R l = G::dot(e, p[lo] - p[i]);
        R r = G::dot(e, p[hi] - p[i]);
        R h = G::cross(e, p[top] - p[i]);
        R area = (r - l) * h / d;
        if (area < best)
        {
            best = area;
            P a = p[i] + e * (l / d);
            P b = p[i] + e * (r / d);
            P v = G::perp(e) * (h / d);
            box = {a, b, b + v, a + v};
        }
    }
    return {best, box};
}
```

正确性 最小面积包围矩形至少有一条边与凸包边共线。固定此方向，左右和上方支撑线由投影最小、投影最大和叉积最大三点决定。严格凸包边方向依次旋转，三个最优支撑点只向前移动；相同投影推进到末端，以跨过平行边。第一条边完整扫描初始化，避免从错误局部极值开始。

恢复公式与边界 设底边方向 e、长度平方 d，最小/最大投影 l,r，最大叉积 h。面积为 (r-l)h/d，底边两端为 p+el/d、p+er/d，上边统一加 perp(e)h/d。不要求首角纵坐标最小。输入必须是严格逆时针凸包；任意实数判向和极薄模型不能由通过有界整数题保证。

模板题使用：P3187 最小矩形覆盖

题意：3≤n≤50000，坐标 [0,10]、至多五位小数。乘 10^5 后四舍五入还原整数，integer_hull 精确去重与判向，再将凸包转 long double 作卡壳。输出面积及逆时针四角，10 位小数；面积误差/max(1,S)<1e-4、点到矩形距离<1e-4。已 100 分 AC，记录 302662813；最慢 26ms，总计 113ms。

所需模板：IntegerPlane（第 19 页，1.11 节）、integer_hull（第 78 页，5.3 节）、RealPlane（第 1 页，1.1 节）、minimum_rectangle（第 86 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3187

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<IntegerPlane::Point> points(n);
    for (auto &p : points)
    {
        long double x, y;
        cin >> x >> y;
        p = {llroundl(x * 100000), llroundl(y * 100000)};
    }
    auto hull = integer_hull(points);
    vector<RealPlane::Point> p;
    for (auto q : hull)
        p.push_back({(long double)q.x, (long double)q.y});
    auto [area, box] = minimum_rectangle(p);
    cout << fixed << setprecision(10) << area / 10000000000.L << '\n';
    for (auto q : box)
        cout << q.x / 100000 << ' ' << q.y / 100000 << '\n';
}
```

第八章　半平面交

8.1　整数半平面的有界多边形交

问题描述：给定若干半平面 $ax+by\le c$（整数系数），已知交集有界，求交集多边形的顶点（逆时针，有理坐标）；交集为空、只是点或线段时返回空。

核心思路：按法向量 (a,b) 的极角排序，法向相同的只保留最紧的一条。用双端队列维护当前交的边界线：加入新半平面前，若队尾（队首）两条线的交点不满足它，就弹出队尾（队首）。全部加入后再用队首、队尾互相清理，相邻两线依次求交得到顶点。交点用 (x,y,d) 表示 $(x/d,y/d)$，所有排序和判定都在 `__int128_t` 中精确完成，不用 eps。

思路中的情况	代码中的处理	行
$0x+0y\le c$	$c\ge 0$ 时恒成立直接丢弃，$c<0$ 时交集为空	53
排序	先按法向半平面 `half`，再按叉积比较极角；法向平行同向时比较归一化的 c，更紧的在前	69
同向平行线	只保留排在前面（最紧）的一条	75
加入新半平面	队尾、队首两线的交点在新半平面外（`value` >0）则弹出	76
与队尾反向平行	交集为空或退化，返回空	79
首尾清理	用队首检查队尾交点、用队尾检查队首交点，删去多余的线	82
输出顶点	相邻两线求交，用有理数相等判断去掉重合顶点	89

接口契约：bounded_polygon(lines) 求 a*x+b*y≤c 的交集闭包顶点，Line={a,b,c}，三个整数系数绝对值均 $\le 10^9$。要求交集有界，空集也允许；只返回正面积多边形，点或线段交集返回空向量。无界输入不在接口范围内。Point={x,y,d} 表示 (x/d,y/d)，d>0，分数不必约分；按逆时针返回，不重复首点，相邻重复点已去除。排序与所有构造、判定使用整数和 int128，不使用 epsilon。O(n log n) 时间、O(n) 空间。浮点转换与面积计算需单独处理。

```
struct IntegerHalfplanes
{
    using I = __int128_t;
    using ll = long long;

    struct Line
    {
        ll a, b, c;
    };

    struct Point
    {
        ll x, y, d;
    };

    static I cross(Line u, Line v) { return I(u.a) * v.b - I(u.b) * v.a; }

    static int half(Line u) { return u.b < 0 || (u.b == 0 && u.a < 0); }

    static Point meet(Line u, Line v)
    {
        I d = cross(u, v);
        assert(d != 0);
        I x = I(u.c) * v.b - I(u.b) * v.c;
        I y = I(u.a) * v.c - I(u.c) * v.a;
        if (d < 0)
        {
            d = -d;
            x = -x;
            y = -y;
        }
        return {(ll)x, (ll)y, (ll)d};
    }
```

接上页同一结构体与函数:

```
35      static I value(Line u, Point p)
36      {
37          return I(u.a) * p.x + I(u.b) * p.y - I(u.c) * p.d;
38      }
39
40      static bool same(Point p, Point q)
41      {
42          return I(p.x) * q.d == I(q.x) * p.d && I(p.y) * q.d == I(q.y) * p.d;
43      }
44
45      static vector<Point> bounded_polygon(vector<Line> lines)
46      {
47          vector<Line> h;
48          for (auto u : lines)
49          {
50              assert(-1000000000 <= u.a && u.a <= 1000000000);
51              assert(-1000000000 <= u.b && u.b <= 1000000000);
52              assert(-1000000000 <= u.c && u.c <= 1000000000);
53              if (!u.a && !u.b)
54              {
55                  if (u.c < 0) return {};
56              }
57              else
58                  h.push_back(u);
59          }
60          sort(h.begin(),
61               h.end(),
62               [](Line u, Line v)
63               {
64                   if (half(u) != half(v)) return half(u) < half(v);
65                   I d = cross(u, v);
66                   if (d) return d > 0;
67                   ll x = u.a ? (u.a > 0 ? u.a : -u.a) : (u.b > 0 ? u.b : -u.b);
68                   ll y = v.a ? (v.a > 0 ? v.a : -v.a) : (v.b > 0 ? v.b : -v.b);
69                   return I(u.c) * y < I(v.c) * x;
70               });
```

接上页同一结构体与函数:

```
        deque<Line> q;
        for (int i = 0; i < (int)h.size(); i++)
        {
            Line u = h[i];
            if (i && half(u) == half(h[i - 1]) && cross(u, h[i - 1]) == 0) continue;
            while (q.size() >= 2 && value(u, meet(q[q.size() - 2], q.back())) > 0)
                q.pop_back();
            while (q.size() >= 2 && value(u, meet(q[0], q[1])) > 0) q.pop_front();
            if (!q.empty() && cross(q.back(), u) == 0) return {};
            q.push_back(u);
        }
        while (q.size() >= 3 && value(q.front(), meet(q[q.size() - 2], q.back())) > 0)
            q.pop_back();
        while (q.size() >= 3 && value(q.back(), meet(q[0], q[1])) > 0) q.pop_front();
        if (q.size() < 3 || cross(q.front(), q.back()) == 0) return {};
        vector<Point> answer;
        for (int i = 0; i < (int)q.size(); i++)
        {
            Point p = meet(q[i], q[(i + 1) % q.size()]);
            if (answer.empty() || !same(answer.back(), p)) answer.push_back(p);
        }
        if (answer.size() > 1 && same(answer.front(), answer.back())) answer.pop_back();
        if (answer.size() < 3) return {};
        return answer;
    }
};
```

第九章　平面最近点对

9.1　浮点平面最近点对

问题描述：给定平面上若干点，求最近两点之间的距离。

核心思路：分治。先按 x 排序，若有重复点直接得 0。`dfs(l,r)` 返回区间内的最近距离，并把区间按 y 排好序：不超过 3 个点时暴力；否则记下中点横坐标，递归两半后按 y 归并。只有离中线不超过当前答案 `best` 的点才可能构成更近的跨中线点对；按 y 顺序把它们放入 `strip`，每个点只向前比较 y 差不超过 `best` 的点，这样的点只有常数个。

接口契约：solve(points) 返回 long double 最短距离（不是平方），少于两个点返回 nullopt，重复点返回零。Point={x,y}；输入须有限，坐标绝对值 ≤1e100。递归分治，按纵坐标归并，O(n log n) 时间、O(n) 额外空间。用 hypotl 求距离，不先平方；排序不使用 epsilon。浮点运算仍有舍入误差，输入前已丢失的精度无法恢复。

```cpp
struct ClosestPair
{
    using R = long double;

    struct Point
    {
        R x, y;
    };

    // Finite coordinates, |x|, |y| <= 1e100. Returns distance, not its square.
    static optional<R> solve(vector<Point> p)
    {
        int n = p.size();
        if (n < 2) return nullopt;
        sort(p.begin(),
             p.end(),
             [](Point a, Point b) { return tie(a.x, a.y) < tie(b.x, b.y); });
        for (int i = 1; i < n; i++)
            if (p[i].x == p[i - 1].x && p[i].y == p[i - 1].y) return R(0);
        auto by_y = [](Point a, Point b)
        {
            return tie(a.y, a.x) < tie(b.y, b.x);
        };
        auto distance = [](Point a, Point b)
        {
            return hypotl(a.x - b.x, a.y - b.y);
        };
        vector<Point> tmp(n);
        function<R(int, int)> dfs = [&](int l, int r) -> R
        {
            if (r - l <= 3)
            {
                R best = numeric_limits<R>::infinity();
                for (int i = l; i < r; i++)
                    for (int j = l; j < i; j++) best = min(best, distance(p[i], p[j]));
                sort(p.begin() + l, p.begin() + r, by_y);
                return best;
            }
```

分治、归并与跨分界线点对（接上页同一函数）：

```
            int m = (l + r) / 2;
            R x = p[m].x;
            R best = min(dfs(l, m), dfs(m, r));
            merge(p.begin() + l,
                  p.begin() + m,
                  p.begin() + m,
                  p.begin() + r,
                  tmp.begin() + l,
                  by_y);
            copy(tmp.begin() + l, tmp.begin() + r, p.begin() + l);
            vector<Point> strip;
            for (int i = l; i < r; i++)
                if (fabsl(p[i].x - x) <= best)
                {
                    for (int j = (int)strip.size() - 1; j >= 0; j--)
                    {
                        if (p[i].y - strip[j].y > best) break;
                        best = min(best, distance(p[i], strip[j]));
                    }
                    strip.push_back(p[i]);
                }
            return best;
        };
        return dfs(0, n);
    }
};
```

模板题使用：Luogu P1429

题意：求平面最近两点的距离，输出小数点后四位。solve 返回距离而非平方；题目保证至少两点，因此可解引用 optional。坐标按 long double 读取。

所需模板：ClosestPair（第 93 页，9.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1429

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<ClosestPair::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    printf("%.4Lf\n", *ClosestPair::solve(p));
}
```

第十章　随机增量法

10.1　随机增量最小覆盖圆

问题描述：求覆盖平面上所有给定点的最小圆（最小覆盖圆）。

核心思路：随机增量法。先随机打乱点序，依次加入 p_i：若它在当前圆外，则前 i 个点的最小圆边界必过 p_i，令圆为点圆 p_i 并重扫前面的点；遇到圆外的 p_j 时边界必过 p_i, p_j，先取以它们为直径的圆，再扫描更早的点，遇到圆外的 p_k 就取三点外接圆。随机顺序下每层重建的概率很小，期望线性。`through` 缩放后解外接圆心，三点共线时退化为最远两点的直径圆；判断点在圆外带相对容差。

代码依赖：RealPlane（第 1 页，1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：solve(points,rng,eps) 返回 optional<Circle>，Circle={o,r}；空集返回 nullopt，单点或重复同一点返回半径零。rng 是调用者提供的 mt19937_64 随机数引擎。期望 O(n) 时间，最坏 $O(n^3)$，O(n) 空间；随机化不可省略。eps 默认 32 倍 long double 机器精度，用于相对半径的包含判断。diameter(a,b) 求直径圆；through(a,b,c) 求外接圆，共线时返回最远点对的直径圆。依赖浮点几何的 Point/Circle 与向量运算。输入及中间运算须有限且尺度适当；接近共线时圆心可能不稳定，无法恢复读入前的精度。

```
struct EnclosingCircle
{
    using G = RealPlane;
    using R = G::R;
    using Point = G::Point;
    using Circle = G::Circle;

    static Circle diameter(Point a, Point b)
    {
        Point o = a + (b - a) / 2;
        return {o, max(G::norm(o - a), G::norm(o - b))};
    }

    static Circle through(Point a, Point b, Point c)
    {
        Point u = b - a, v = c - a;
        R scale = max({fabsl(u.x), fabsl(u.y), fabsl(v.x), fabsl(v.y)});
        if (scale == 0) return {a, 0};
        u = u / scale;
        v = v / scale;
        R det = 2 * G::cross(u, v);
        if (det == 0)
        {
            Circle best = diameter(a, b);
            for (Circle q : {diameter(a, c), diameter(b, c)})
                if (q.r > best.r) best = q;
            return best;
        }
        R x = G::dot(u, u), y = G::dot(v, v);
        Point offset{(x * v.y - y * u.y) / det, (u.x * y - v.x * x) / det};
        Point o = a + offset * scale;
        return {o, max({G::norm(o - a), G::norm(o - b), G::norm(o - c)})};
    }
```

随机增量主过程（接上页同一结构体）：

```
    static optional<Circle>
    solve(vector<Point> p, mt19937_64 &rng, R eps = 32 * numeric_limits<R>::epsilon())
    {
        assert(eps >= 0 && eps < 1);
        if (p.empty()) return nullopt;
        shuffle(p.begin(), p.end(), rng);
        Circle c{p[0], 0};
        auto outside = [&](Point q)
        {
            return G::norm(q - c.o) > c.r * (1 + eps);
        };
        for (int i = 1; i < (int)p.size(); i++)
            if (outside(p[i]))
            {
                c = {p[i], 0};
                for (int j = 0; j < i; j++)
                    if (outside(p[j]))
                    {
                        c = diameter(p[i], p[j]);
                        for (int k = 0; k < j; k++)
                            if (outside(p[k])) c = through(p[i], p[j], p[k]);
                    }
            }
        return c;
    }
};
```

模板题使用：Luogu P1742

题意：求包含全部点的最小圆，先输出半径，再输出圆心坐标。题目保证非空，因此可解引用返回值；显式传随机引擎，不能省略随机打乱。用 long double 读取并输出十二位小数。

所需模板：RealPlane（第 1 页，1.1 节）、EnclosingCircle（第 96 页，10.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P1742

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<EnclosingCircle::Point> p(n);
    for (auto &v : p) scanf("%Lf%Lf", &v.x, &v.y);
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    auto c = *EnclosingCircle::solve(p, rng);
    printf("%.12Lf\n%.12Lf %.12Lf\n", c.r, c.o.x, c.o.y);
}
```

第十一章　配套知识与验证范围

第十二章　计算几何：精确判定与适用范围

12.1　定点凸包、直径和极角排序

两位小数坐标按字符串读成放大 100 倍的整数再求整数凸包，不要读浮点后截断。

旋转卡壳输入须为严格逆时针凸包，不重复首点、不含共线边内点；用整数叉积比较面积，仅在面积严格增大时推进对踵点，空集、单点和线段单独处理。返回 128 位距离平方。

整数极角比较器从正 x 轴按 $[0, 2\pi)$ 排序，同向按距离递增，零向量最前：先分半圆，再比精确叉积，最后比长度；坐标轴归属必须明确，不能用 epsilon 相等作排序等价关系。要求 $(-\pi, \pi]$ 时，先按库排序，再把 $y < 0$ 后缀循环移到最前（负 x 轴为 π，仍在最后）。

12.2　整数系数的有界半平面交

实现见第 89 页。输入 $ax + by \le c$，系数绝对值不超过 10^9 的整数。要求交集有界，只输出正面积多边形；无解、点交和线段交都返回空向量，空返回不等于闭交集为空。无界输入不在约定内。

两条非平行边界交点用齐次坐标

$$D = a_1 b_2 - b_1 a_2, \quad X = c_1 b_2 - b_1 c_2, \quad Y = a_1 c_2 - c_1 a_2,$$

若 $D < 0$ 三者同取负；用 $aX + bY \le cD$ 判断在半平面内。$|X|, |Y|, D \le 2 \times 10^{18}$ 存 64 位，比较的交叉乘积至多 4×10^{36} 用 128 位。按法向半圆与叉积排序，同向平行约束保留更紧者；零法向约束先单独处理。排序后单调队列线性扫描，总时间 $O(n \log n)$。逆时针边 $(x, y) \to (u, v)$ 转为 $a = v - y, b = x - u, c = ax + by$（左侧为可行域）。鞋带项分子 $X_i Y_j - Y_i X_j$、分母 $D_i D_j$ 先以 128 位求值再转浮点。

12.3　最近点对、Minkowski 和与三维整数判定

Minkowski 和见第 79 页。输入为严格逆时针凸包，不重复首点、不含共线冗余点，允许任意起点及空集、单点、线段；输入坐标及任意两点坐标和须在 $[-10^{12}, 10^{12}]$ 内。非退化时按边方向归并，$O(n + m)$；退化时枚举点和再求凸包，$O((n + m) \log(n + m))$。返回同样约定的严格凸包。

三维判定见第 60 页，坐标绝对值不超过 10^9。`orient(a,b,c,d)` 返回有向六倍四面体体积，零表示共面，交换两点变号；`collinear` 判三点共线，`on_segment` 含端点并支持零长度线段。点差分量至多 2×10^9，叉积分量至多 8×10^{18}，三重积至多 4.8×10^{28}，在 int128 内；公开的叉积、点积辅助函数只对点差及其叉积保证不溢出。不含三维凸包构造。

12.4　浮点最近点对

实现见第 93 页。`ClosestPair::solve` 接收 `Point{x,y}` 数组，返回最短距离（不是平方）；少于两点返回空 optional，重复点返回零。输入为有限 `long double`，坐标绝对值不超过 10^{100}。分治按纵坐标归并，$O(n \log n)$ 时间、$O(n)$ 额外空间；用 `hypotl`，排序不用 epsilon。

12.5　公切线与过点切线

实现见第 30 页。令圆心差 v，$d = |v|$，$u = v/d$；外公切线 $s = 1$，内公切线 $s = -1$：

$$n \cdot u = \frac{r_1 - s r_2}{d} = c, \qquad n = cu \pm \sqrt{(1 - c)(1 + c)}\, u^{\perp}.$$

切点 $p = o_1 + r_1 n$、$q = o_2 + s r_2 n$，直线为 $n \cdot (x - p) = 0$。`side` 为 $1/-1$ 表示外/内公切线；非相切分支两条，相切只输出一次。半径零视为点：一个点圆时只枚举外分支；两不同点返回连线，`side=0`；同心等半径返回 `infinite=true`，同心不等半径无公切线。`from_point(p,c)` 把点设为第一圆，圆上切点读 `b`；点在圆上一条，圆内为空。输入有限，坐标和半径绝对值不超过 10^{100}，半径非负。容差 $0 < \varepsilon < 1$ 默认 10^{-12}：$|r_1 - s r_2|$ 与 d 相对差在容差内即合并为相切，近切输入不承诺恢复精确分类。

12.6 随机增量最小覆盖圆

实现见第 96 页。随机打乱后依次加入；圆外点必为新圆边界点，依次固定一、二、三个边界点重扫前缀。期望 $O(n)$，最坏 $O(n^3)$，无递归。必须传入初始化好的 `mt19937_64`，不可省略打乱。空集返回空 optional，单点或重复点半径为零；三点共线时取最远点对的直径圆。`through` 只求外接圆，不是最小覆盖圆。包含判断用相对半径容差，默认 32 倍机器精度；近共线三点外接圆数值敏感。

12.7 圆与简单多边形相交面积

实现见第 28 页。圆心移到原点，沿有向边累加，最后取绝对值。边被交点分成至多三段：小段中点在圆内贡献 $\operatorname{cross}(p,q)/2$，否则贡献有向扇形

$$\frac{r^2}{2}\operatorname{atan2}\big(\operatorname{cross}(p,q), p \cdot q\big).$$

必须保留符号，不能逐边取绝对值。交点取 $h = \operatorname{cross}(u,a)$（$u$ 为单位边向），

$$hu^{\perp} \pm u\sqrt{(r-|h|)(r+|h|)}.$$

$O(n)$ 时间、$O(1)$ 额外空间。多边形须简单，可凹、任意方向、允许相邻重复顶点；半径非负，零半径或少于三个顶点面积为零。极小相交面积不保证相对误差。

12.8 最近点对编号与多边形闭集查询

整数坐标至 10^{12} 时最近点对距离平方可达 8×10^{24}，需用 int128；返回原下标时内部排序下标，不改变输入。

一般简单多边形直接调用 `polygon_contains`，不要先变成凸包；返回 0/1/2 为外部、边界、内部。凸包二分查询要求严格逆时针、无共线边内点、不重复首点。

两组点凸包 A, B，第二组平移 d 后相交当且仅当 $d \in A + (-B)$：先对第二组取负再求 Minkowski 和，对原平移向量做闭集查询（查询不取负，边界算相交）。复杂度 $O(n \log n + m \log m + q \log(n+m))$。

12.9 整数圆交点：精确判定与浮点构造

整数坐标、半径绝对值不超过 10^9，半径可为零。直线：$v = b - a$，$q = v \cdot v$，$c = v \times (a - o)$，$D = r^2 q - c^2$；$q = 0$ 为退化直线，否则按 D 符号分类；垂足 $h = o + \operatorname{perp}(v) c/q$，交点 $h \pm v\sqrt{D}/q$。两圆：$v = b - a$，$q = v \cdot v$，$k = q + r^2 - s^2$，$D = 4r^2 q - k^2$，交点 $a + vk/(2q) \pm \operatorname{perp}(v)\sqrt{D}/(2q)$；$q = 0$ 同心单独处理：相同正半径无穷交点，相同零半径一点。$q, |c| \le 8 \times 10^{18}$，$|k| \le 9 \times 10^{18}$，平方项至多 8.1×10^{37}，须先提升到 int128 再运算；扩大范围需重新核算。只有分类精确，坐标仍有舍入误差，在 10^9 尺度不承诺绝对 10^{-6}。相切只返回一个点；结果按精确字典序排序，不用 eps 比较器。

12.10 公切线：切点次序也需要精确判定

`CircleTangents` 的 `line.a`、`line.b` 分别是第一、第二圆的切点；`from_point` 把给定点视为第一圆，切点读 `line.b`。按切点坐标排序时浮点会错排相等横坐标，应精确比较。

圆心差 $v = (x, y)$，$q = x^2 + y^2$，外、内公切线取 $g = r - s$、$g = r + s$，$D = q - g^2$；$D < 0$ 无此类线，$D = 0$ 一条，否则两条。单位法向 $n = (vg \pm \operatorname{perp}(v)\sqrt{D})/q$；第一圆半径为正时，比较切点等价于比

较法向分量。比较化为 $A+B\sqrt{C}+E\sqrt{F}$ 的符号：令 $u=A+B\sqrt{C}$、$w=E\sqrt{F}$，同号直接决定；异号时求 $u^2-w^2=(A^2+B^2C-E^2F)+2AB\sqrt{C}$ 的符号再乘 $\operatorname{sgn}(u)$。单根式也先判符号再平方。坐标、半径至 10^4 时中间量不超过 2.304×10^{35}，int128 可容纳；不能沿用圆交点的 10^9 范围。相切时用法向定义唯一切线，不用两个重合切点求方向。

12.11 圆交点：容差分类与坐标误差

浮点实现见第 6、8 页。默认按相对容差处理近切，距离误差小不推出交点坐标误差小。显式传 `eps=0` 使用无容差分支，但不等于任意实数上的精确几何。

避免先开方再相减。 直线与两圆分别用上节的 $q,c,\Delta=r^2q-c^2$ 和 $q,k,\Delta=4r^2q-k^2$；直线偏移 $w=v\sqrt{\Delta}/q$，两圆 $h=a.o+vk/(2q)$、$w=\operatorname{perp}(v)\sqrt{\Delta}/(2q)$，返回 $h\pm w$。

AOJ 整数输入域的额外保证。 坐标绝对值不超过 $B=10^4$、半径 $1..B$ 的整数时，所有中间整数小于 $81B^4<2^{60}$；`long double` 有效尾数至少 64 位、正常舍入且不启用 fast-math 时，判别式符号精确。53 位尾数不够。

构造与排序。 上述条件下交点每坐标误差小于 1.1×10^{-14}，不同交点非零横坐标间隔至少 1.25×10^{-9}，故可直接按 (x,y) 字典序排序，不用 epsilon 比较器。

范围之外。 其他实数和更大尺度上判别式仍可能消去、上溢或下溢；`RealGeometry` 构造仍要求正 eps。

12.12 三维投影、旋转与平面求交的数值约定

三维向量、直线与线段、平面分别见第 61、65、68 页。

`Line3(p,d)` 第二参数是方向，表示 $p+td$；端点构造用 `Line3::through(a,b)`。投影到直线不截断，到线段限制 $0\le t\le1$。零方向按点处理，不能作旋转轴。向量排序用精确字典序。单位轴 u、$v=q-p$ 的右手旋转（弧度）：

$$p+v\cos\theta+(u\times v)\sin\theta+u(u\cdot v)(1-\cos\theta).$$

`angle(a,b)` 返回 $[0,\pi]$；无向平面夹角取 $\min(\theta,\pi-\theta)$。

平面存点 p 与单位法向 n：$n\cdot(x-p)=0$。`equation(n,h)` 表示 $n\cdot x=h$，一般式 $Ax+By+Cz+D=0$ 传 $h=-D$；法向非零，三点不共线。有符号距离 $n\cdot(q-p)$，投影 $q-n[n\cdot(q-p)]$。求交返回 1 单点或交线、0 无交、-1 重合或线在面上；仅返回 1 时写输出参数。两平面非平行时取 $d=\operatorname{unit}(n\times m)$、$v=d\times n$，

$$q=p+v\frac{m\cdot(f.p-p)}{m\cdot v},\qquad \text{交线为}q+td,$$

分母不能取绝对值。默认距离容差 10^{-10}、角度正弦容差 10^{-12}；近平行分类依赖尺度与容差，需要精确共面谓词时用第 60 页的整数版本。

12.12.1 原三维片段中需要避开的错误

坐标须有限，中间量避免溢出和下溢；平面角为 $\arccos\big((n\cdot m)/(|n||m|)\big)$，闭线段判定用点积范围而非等于零。巨大平移与微小差值、近共线和近平行求交会放大误差，应先平移/缩放并按题意选容差；不能假定 `long double` 总比 `double` 精确。

算法索引

circle_coverage_area, 44
circle_intersections, 8
circle_intersections_i64, 26
circle_overlap_area, 11
circle_union_area, 39
CirclePolygon, 28
CircleTangents, 30
ClosestPair, 93
convex_contains_i64, 77
convex_cut, 12
convex_diameter2, 84
convex_tangents_i64, 74

EnclosingCircle, 96
Euler characteristic, 73

Hull3D, 49

integer_hull, 78
IntegerGeometry3D, 60
IntegerHalfplanes, 89
IntegerPlane, 19

lattice polygon, 73
Line3, 65
line_circle_i64, 24
line_circle_intersections, 6
line_intersection_real, 4
line_projection, 3

minimum_rectangle, 86
minkowski_sum, 79

Pick theorem, 73
Plane3, 68
polygon_area2, 22
polygon_centroid, 14
polygon_contains, 23
polygon_union_area, 32

RealPlane, 1
RealPolarLess, 16
RealSpace, 61
rectangle_union_area, 82

segment_distance_real, 71

triangle_incircle, 15